

## 查找和排序

### 查找表

#### 线性表

##### 顺序查找

##### 算法

##### 算法分析

##### 如何提高查找效率

##### 顺序查找特点

#### 折半查找(Binary Search)

##### 算法分析--判断树

##### 折半查找特点

#### 分块查找(Blocking Search)

##### 算法分析

##### 分块查找特点

#### 线性表查找方法比较

### 树表

#### 二叉排序树(Binary Sort Tree)

##### 数据类型定义

##### 复习：递归创建二叉树

##### BST查找

##### BST查找算法分析

##### BST插入

##### 创建BST

##### BST删除

#### AVL树(Balance Binary Tree)

##### LL型旋转(右旋转)

##### RR旋转(左旋转)

##### LR旋转(左右旋转)

##### RL旋转(右左旋转)

##### 总结

##### 代码实现

##### 左旋转和右旋转(参照LL，RR旋转)

##### 左平衡和右平衡

##### 插入节点和及时平衡

##### 测试代码

#### 哈希表(Hash Table)

#### hash函数的构造方法

##### 处理冲突的方法

##### 除留余数法构造Hash Table

##### 查找效率分析

##### 几点结论

### 查找总结

### 排序

#### 直接插入排序

#### 折半插入排序(Binary Insert Sort)

#### 希尔排序(Shell's Sort)

#### 交换排序

##### 冒泡排序

##### 快速排序

选则排序

堆

堆定义

定理1:

定理2:

初始化堆

堆调整

堆排序

其他类型排序

并归排序 Merge Sort

合并两个有序序列

分割序列并进行合并

函数封装

算法效率

基数排序

数据类型定义

分配函数

收集函数

排序函数

初始化和输出函数

算法效率

排序总结

# 查找和排序

## 查找表

### 关键字

- 主关键字：可唯一标识一个记录的关键字
- 次关键字：用以识别若干记录的关键字

### 查找表分类

- 线性表
- 树表
- 哈希表

### 动态和静态

- 静态：仅作查询，检索
- 动态：作插入和删除操作

### 平均查找长度ASL(Average Search Length)

- 关键字的平均比较次数

$$ASL = \sum_{i=1}^n p_i c_i$$

$n$  : 记录的个数(表长)

$p_i$  : 查找第 $i$ 个元素的概率

$c_i$  : 查找到第 $i$ 个元素需要的比较次数

## • 线性表

### - 顺序查找

#### 应用范围

- 顺序表或线性链表的静态查找
- 表内元素无序

#### 类型定义:

```

typedef int KeyType;
typedef char *OtherInfo;
typedef struct {
    KeyType key;
    OtherInfo other; //储存其他信息
} SSTElemType;
typedef struct {
    SSTElemType *base; //存放数组的首地址(0号位置不存放关键字, 预留guard)
    int length; //当前表长(数组的长度)
} SSTable;

```

## - 算法

思路:把base数组中0号位置预留出来, 从最后一个元素出发, 依此向0号方向比较

```

int Search_SS(SSTable ST, KeyType key) {
    for (int i=ST.length; i>0; i--){
        if (key == ST.base[i].key) return i;
    }
    return 0;
}

```

可发现此算法中一次循环 `i>0;` 和 `key == ST.base[i]` 比较了两次

其他形式:

```

int Search_SS(SSTable ST, KeyType key) {
    int i;
    for (i = ST.length; i>0 && ST.base[i].key != key; i--); //注意;

    return i;
}

```

`i>0 && ST.base[i].key != key` 同样的比较了两次

优化算法

为数组中0号位置添加guard

```

int Search_SS(SSTable ST, KeyType key) // Sequence Search
{
    ST.base[0].key = key; //预留0号位置, 设置guard
    int i;
    for (i = ST.length; ST.base[i].key != key; i--); //分号不可以丢

    return i;
}

```

可发现循环中 `ST.base[i].key!= key;` 只比较一次



## - 算法分析

| 0  | 1 | 2 | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 |
|----|---|---|----|----|----|----|----|----|----|----|----|
| 哨兵 | 5 | 7 | 19 | 21 | 13 | 56 | 64 | 92 | 88 | 80 | 75 |

观察此图，发现想查到第 $i$ 个位置，需要 $n - i + 1$ 次比较

查找失败则需要 $n + 1$ 次比较

$$\text{令 } p_i = \frac{1}{n}$$

$$ASL = \frac{1}{n} \sum_{i=1}^n n - i + 1 = \frac{n+1}{2}$$

所以 $O(n)$

时间复杂度： $O(n)$

空间复杂度(需要额外的数组中0号位置)： $O(1)$

## - 如何提高查找效率

### 1. 按查找概率高低存储

- 查找概率高，比较次数少
- 查找概率低，比较次数多

### 2. 当查找概率无法确定时

- 按查找概率动态调整
- 在每个key中增设一个访问频度域
- 始终保持频度域按非递增，有序的次序排列
- 每次查找后，讲刚查到的key数据移至表头

## - 顺序查找特点

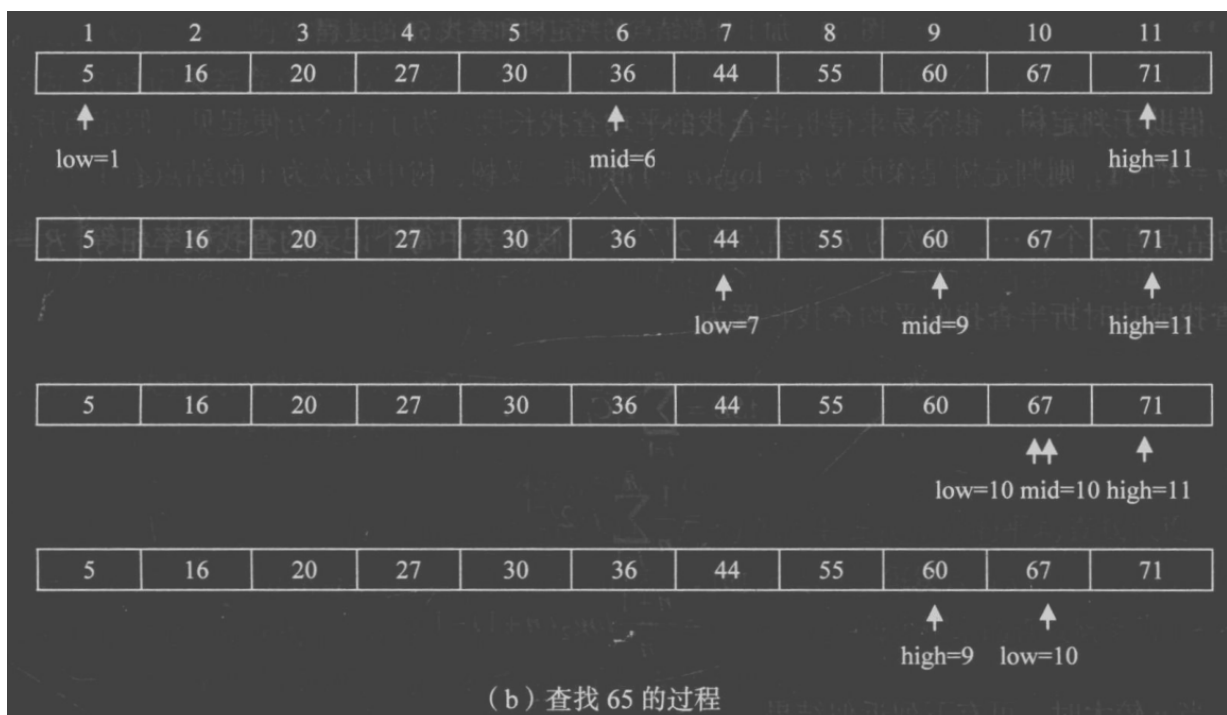
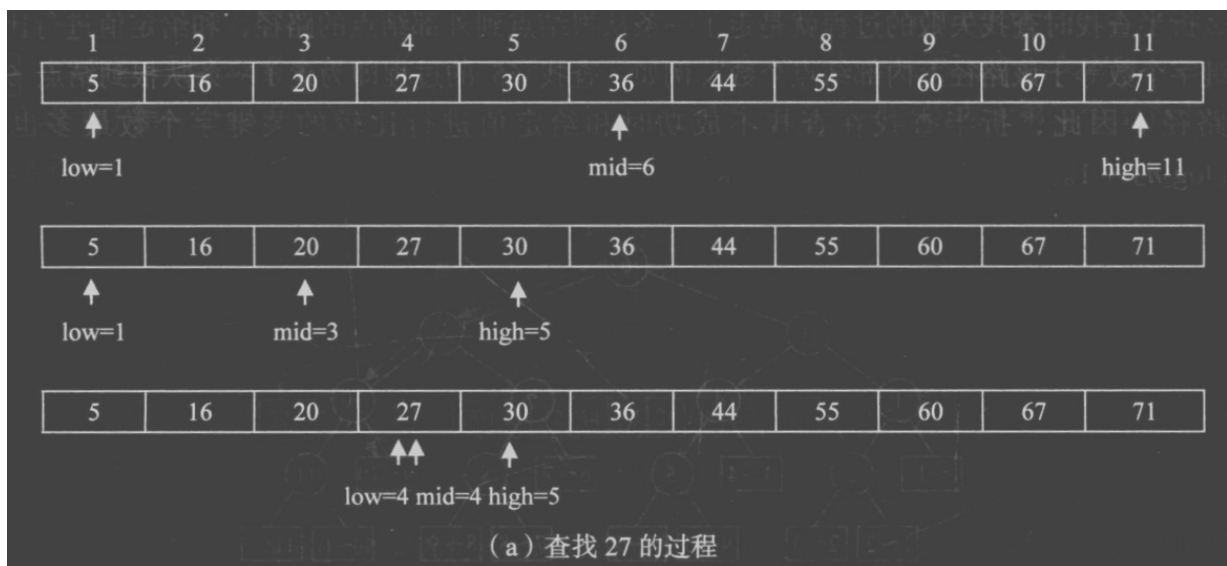
- 优点：算法简单，逻辑次序无要求，不同存储结构均适用
- 缺点：ASL太长，时间效率低

## • 折半查找(Binary Search)

前置条件：集合中的元素按递增的顺序排列

方法：每次将集合中的元素缩小一半

需要变量 low,high或left,right



由图可知，如果查找的元素不在集合中，那么最终的结果为  $low > high$ ，即循环的条件为  $while(low \leq high)$

```
int Search_BS(SSTable ST, KeyType key) {
    int left = 1;           //确定左区间
    int right = ST.length;  //确定右区间
    int mid = (left + right) / 2;

    while (left <= right) {
        if (key == ST.base[mid].key)
            return mid;
        else if (key > ST.base[mid].key) { //如果搜索元素大于中间位置元素
            left = mid + 1;
            mid = (left + right) / 2;
        } else { //如果搜索元素小于中间位置
            right = mid - 1;
            mid = (left + right) / 2;
        }
    }

    return 0;
}
```

}

## - 算法分析--判断树

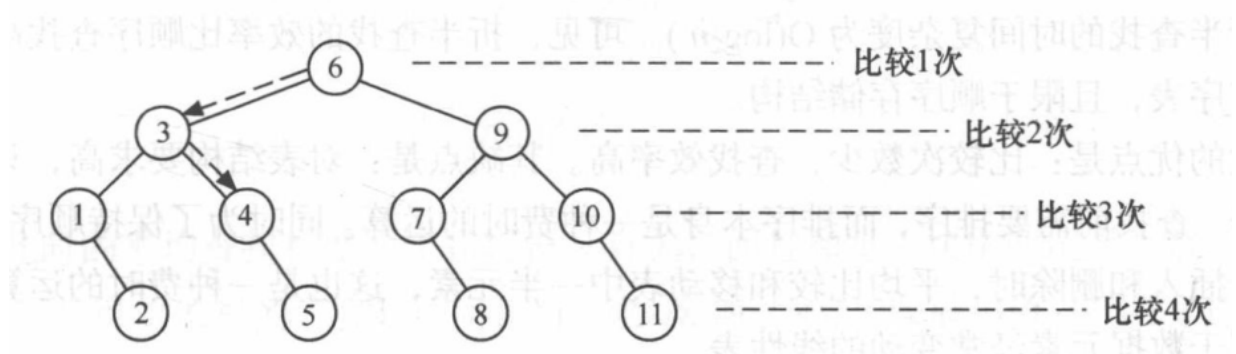
二叉树性质

节点数 :  $n = 2^h - 1$

高度 :  $h = \log_2^{n+1} = [\log_2(n)] + 1$

第 $n$ 层节点个数 :  $2^{n-1}$

可以把该搜索过程抽象成一颗二叉树，在数组中的位置表示节点



可以看出该二叉树的 $height$ 为4

那么该二叉树的节点数为 :  $2^{h-1} - 1 < n \leq 2^h - 1$ , 即  $7 < n \leq 15$

由此可见，找到元素所在位置经过的路径就是需要的比较次数

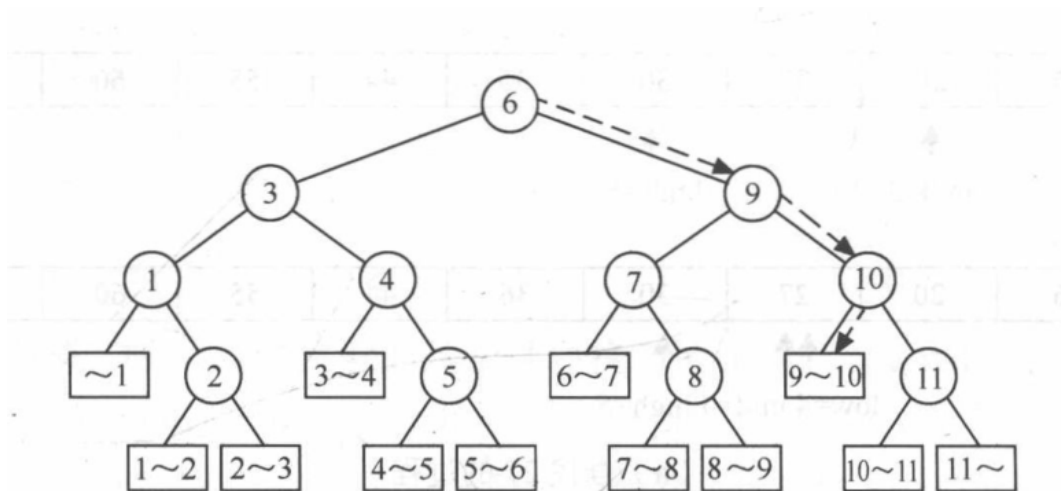


图 7.3 加上外部结点的判定树和查找 65 的过程

方形节点代表着外部节点，如果到达方形节点则表示查找失败，即  $low > high$

为了方便讨论，设一个数组中有  $n = 2^h - 1$  个元素

$$ASL = \sum_{i=1}^h c_i p_i$$

令每个元素找到的概率相等  $p_i = \frac{1}{n}$

$$ASL = \frac{1}{n} \sum_{i=1}^n = \frac{1}{n} \sum_{k=1}^h k 2^{k-1} (\text{转化成高度}) = \frac{n+1}{n} \log_2^{(n+1)} - 1$$

$$\text{Note: } \sum_{k=1}^n k 2^{k-1} = (k-1)2^k + 1$$

$$\text{推导: } \sum_{k=1}^n k 2^{k-1} = (k-1)2^k + 1$$

假设有  $\{a_n\} = n \cdot 2^{n-1}$ ，则有数列 1, 4, 12, 32...

令  $\{b_n\} = a_{n+1} - a_n$ ，则有数列 3, 8, 20, ...

$$b_n = (n+1) \cdot 2^n - n \cdot 2^{n-1} = 2^{n-1} \cdot (n+2)$$

$$\begin{aligned} \sum_{k=1}^n b_k &= \sum_{k=1}^n 2^{k-1} \cdot (k+2) \\ &= \sum_{k=1}^n k \cdot 2^{k-1} + 2^n \\ &= \sum_{k=1}^n k \cdot 2^{k-1} + \sum_{k=1}^n 2^n \end{aligned}$$

由  $\{b_n\} = a_{n+1} - a_n$  可知：

$$\begin{aligned} \sum_{k=1}^n a_{k+1} - a_k &= a_{n+1} - a_1 \\ &= \sum_{k=1}^n k \cdot 2^{k-1} + \sum_{k=1}^n 2^n \\ &= \sum_{k=1}^n b_k \end{aligned}$$

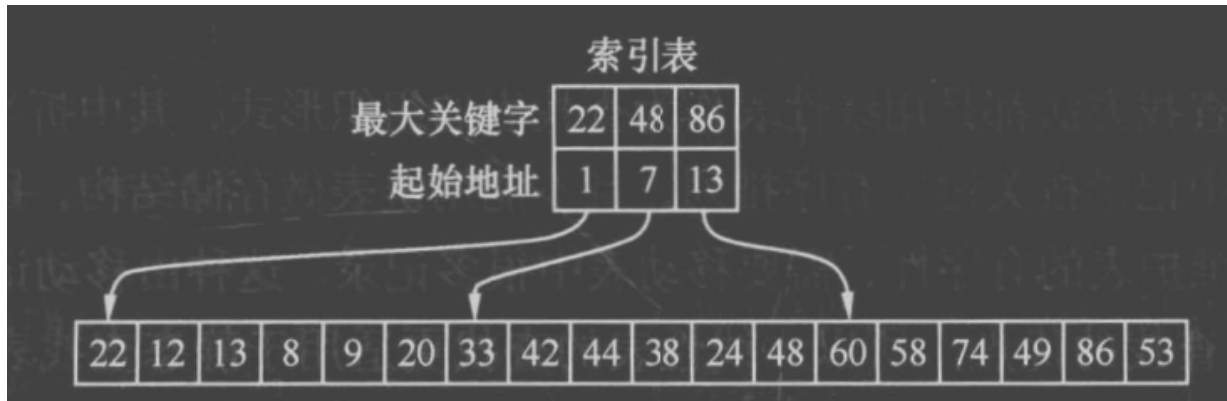
$$\begin{aligned} \sum_{k=1}^n k \cdot 2^{k-1} &= a_{n+1} - a_1 - \sum_{k=1}^n 2^n \\ &= (n+1) \cdot 2^n - 1 - 2^{n+1} + 2 \\ &= (n-1) \cdot 2^n + 1 \end{aligned}$$

## 折半查找特点

- 优点：效率比顺序查找高
- 缺点：只适用于有序表，且仅限于顺序存储结构(对链表无效)

## • 分块查找(Blocking Search)

将表分为几块，并且分块有序，块内可以无序，如下图



把所有抽象成一个数组，数组内元素是最大关键字，对块进行折半查找，对块内进行顺序查找

数据类型定义：

```
#define MAXBLOCK 20
typedef struct {
    KeyType MaxKey;
    int start, end;
} IndexElemType;

typedef struct __IndexTable {
    IndexElemType index[MAXBLOCK];
    int length;
} IndexTable;
```

算法实现

```
IndexTable INDEXTABLE; //全局变量索引表，调用BlockSearch的时候需要extern声明
int BlockSearch(KeyType *a, KeyType key) //此函数，默认不使用数组中0号下标，从数组下标1开始
{
    int left = 1;
    int right = INDEXTABLE.length;

    while (left <= right) {
        int mid = (left + right) / 2;
        if (key <= INDEXTABLE.index[mid].MaxKey) { //如果小于mid,则需要判断是否大于mid-1
            if (key > INDEXTABLE.index[mid - 1].MaxKey) { //如果大于mid-1 说明在mid所在块
                for (int i = INDEXTABLE.index[mid].start; i <= INDEXTABLE.index[mid].end; i++) {
                    if (key == a[i])
                        return i; //进行顺序搜索
                }
                return 0; //没找到返回0
            } else { //小于等于mid-1 则需要进行下次的折半查找
                right = mid - 1;
            }
        } else { // key大于mid, 进行下一次折半查找
            left = mid + 1;
        }
    }
}
```

```

    }
}
return 0; // while循环后依旧没有return, 说明没有找到, 返回0
}

```

## 测试案例

```

#include "SSTable.h"
extern IndexTable INDEXTABLE;

int main(void) {
    KeyType a[19] = {0, 22, 12, 13, 8, 9, 20, 33, 42, 44,
                    38, 24, 48, 60, 58, 74, 49, 86, 53};
    //不计入0号元素, 初始化时0号位置为0
    INDEXTABLE.length = 3;
    INDEXTABLE.index[1].start = 1, INDEXTABLE.index[1].end = 6,
    INDEXTABLE.index[1].MaxKey = 22;
    INDEXTABLE.index[2].start = 7, INDEXTABLE.index[2].end = 12,
    INDEXTABLE.index[2].MaxKey = 48;
    INDEXTABLE.index[3].start = 13, INDEXTABLE.index[3].end = 18,
    INDEXTABLE.index[3].MaxKey = 86;
    printf("%d\n", BlockSearch(a, 86));
    system("pause");
    return 0;
}
/*
22 12 13 8 9 20 33 42 44 38 24 48 60 58 74 49 86 53
*/

```

## 算法分析

$L_b$ 为对 $indextable$ 进行的折半查找,  $L_w$ 为对块内进行的顺序查找,  $s$ 为每一块内元素个数

$$ASL_{bs} \approx \log \left( \frac{n}{s} + 1 \right) + \frac{n+1}{2}$$

$$\log n \leq ASL_{bs} \leq \frac{n+1}{2}$$

例如当 $n = 9, s = 3$ 时

$ASL_{bs} = 3.5$ , 折半查找为3.1, 顺序查找为5

## 分块查找特点

优点: 插入和删除比较容易, 无需进行大量移动

缺点: 要增加一个 $indextable$ 数组(索引表)的储存空间并对初始索引进行排序运算

使用情况: 如果线性表要快速查找且又经常动态变化, 则可采用分块查找

## • 线性表查找方法比较

|      | 顺序查找       | 折半查找 | 分块查找      |
|------|------------|------|-----------|
| ASL  | 最大         | 最小   | 适中        |
| 结构   | 有序表, 无需表OK | 仅有序表 | 分块有序      |
| 存储结构 | 循序表, 链表OK  | 链表NO | 顺序表, 链表OK |

## • 树表

当表插入, 删除操作频繁时, 为维护表的有序性, 需要移动表中很多记录, 有一种方法就是改用动态查找表--树表

对于给定的key值, 若表中存在则成功返回, 若不存在则插入一个等于key值的记录

树表分为:

- 二叉排序树
- 平衡二叉树
- 红黑树
- B-树
- B+树
- 建树

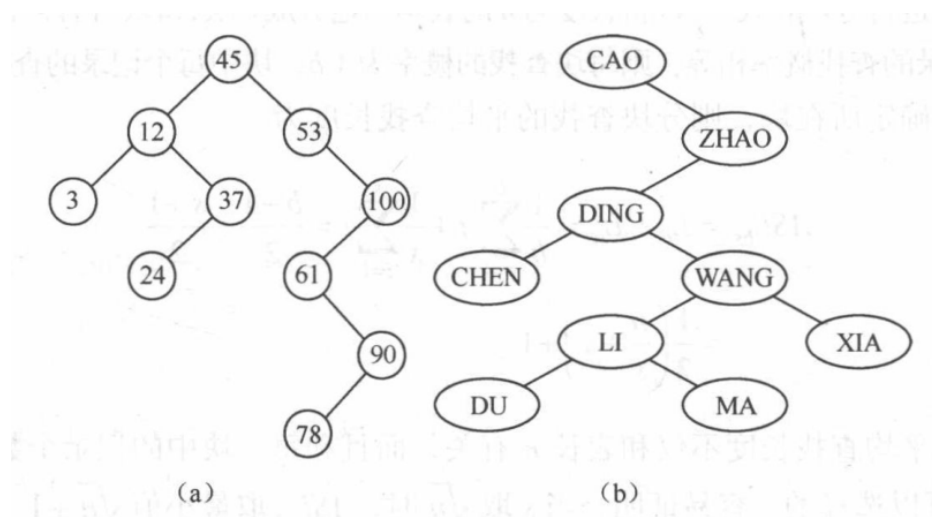
此笔记暂时只记录**二叉排序树**和**平衡二叉树**

## • 二叉排序树(Binary Sort Tree)

定义:

- 若其左子树非空, 则左子树上所有节点的值均小于根节点的值
- 若其右子树非空, 则右子树上所有节点的值均大于等于根节点的值
- 其左右子树本身又是一颗二叉排序树

如下图



如果中序遍历非空二叉排序树，所得到的元素数据序列是一个递增有序数列

## - 数据类型定义

```
#define ENDFLAG 0 //输入结束符
typedef int BSTKeyType;
typedef char *BSTOtherInfo;
typedef struct __BSTElemType {
    BSTKeyType key;
    BSTOtherInfo other;
} BSTElemType;
typedef struct __BSTNode {
    BSTElemType data;
    __BSTNode *lchild, *rchild;
} BSTNode, *BSTree;
```

## - 复习：递归创建二叉树

可以按照Pre-order或Post-order创建二叉树，但是无法按照In-order创建。

这是因为在Pre和Post中连续的虚空节点可以确定唯一的叶子节点，但是In-order不能。

```
Status CreatBTree(BSTree *T) {
    BSTKeyType key;
    scanf(" %d", &key);
    if (0 == key)
        *T = NULL;
    else {
        if (!( *T = (BSTree)malloc(sizeof(BSTNode))))
            exit(OVERFLOW);
        (*T)->data.key = key;
        CreatBTree(&(*T)->lchild);
        CreatBTree(&(*T)->rchild);
    }
    return OK;
}
```

## - BST查找

若关键字等于根节点，成功

否则：

- 若小于根节点，查其左子树
- 若大于等于根节点，查起右子树
- 依此递归



```

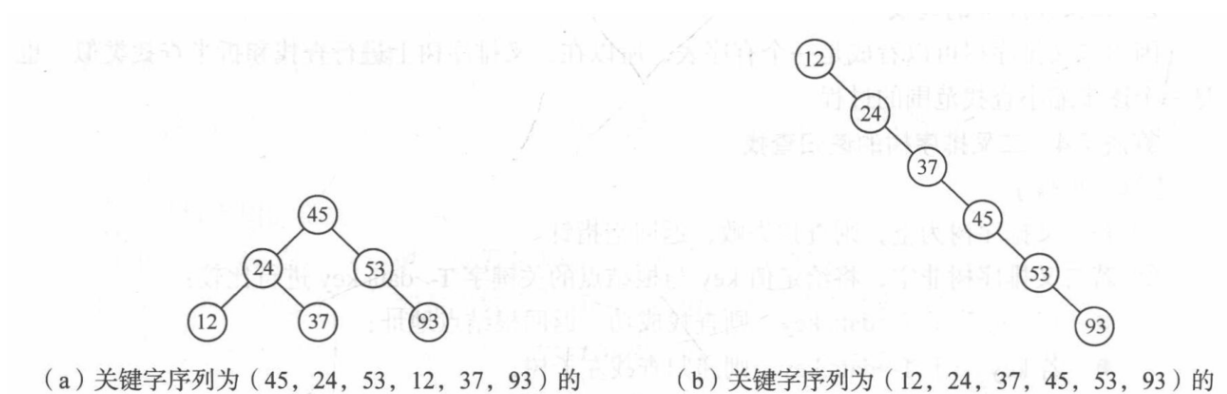
BSTree Search_BST(BSTree T, BSTKeyType key) {
    if (!(T) || key == T->data.key)
        return T;
    else if (key < T->data.key)
        return Search_BST(T->lchild, key);
    else
        return Search_BST(T->rchild, key);
}

```

## - BST查找算法分析

BST查找算法类似于折半查找，每个节点的比较次数和所在层次有关，所及 $O(\log_2^n)$

含有n个节点的二叉排序树的平均查找长度与此二叉树的形态有关，如下图



节点数一样，但ASL显然不同

(a)的 $ASL \approx$  折半查找 =  $O(\log 2)$

(b)的 $ASL =$  顺序查找 =  $O(n)$

所以说在创建二叉排序树的时候，**尽量要让此二叉树形状均匀**

## - BST插入

BST插入算法和线性表的插入算法不同，不需要具体位置

```

void Insert_BST(BSTree *T, BSTKeyType e) {
    if (NULL == *T) {
        //如果当前节点为空，则表示找到合适位置，创建新节点
        *T = (BSTree)malloc(sizeof(BSTNode));
        (*T)->data.key = e;
        (*T)->lchild = (*T)->rchild = NULL;
    } else if (e < (*T)->data.key) {
        //表示位置在T树的左子树
        Insert_BST(&((*T)->lchild), e);
    } else if (e > (*T)->data.key) {
        //在T树的右子树
        Insert_BST(&((*T)->rchild), e);
    }
}

```

## 测试代码

```
#include "BSTree.h"

int main(void) {
    BSTree T = NULL;
    CreatBSTree(&T); //以先序遍历的顺序创建二叉树，0表示为虚空节点
    Insert_BST(&T, 66);
    InOrder_BST(T);
    system("pause");
    return 0;
}

//// 45 12 3 0 0 37 24 0 0 0 53 0 100 61 0 90 78 0 0 0 0
```

## - 创建BST

若从一颗空树T出发，依次插入节点，那么可以创建一个二叉排序树

```
void Creat_BST(BSTree *T) {
    *T = NULL; //初始化为NULL，以便Insert—BST可以运行
    BSTKeyType key;
    scanf("%d", &key);
    while (key != ENDFLAG) { // ENDFLAG 为 0
        Insert_BST(T, key);
        scanf("%d", &key); //注意：while内部必须要有输入
    }
}
```

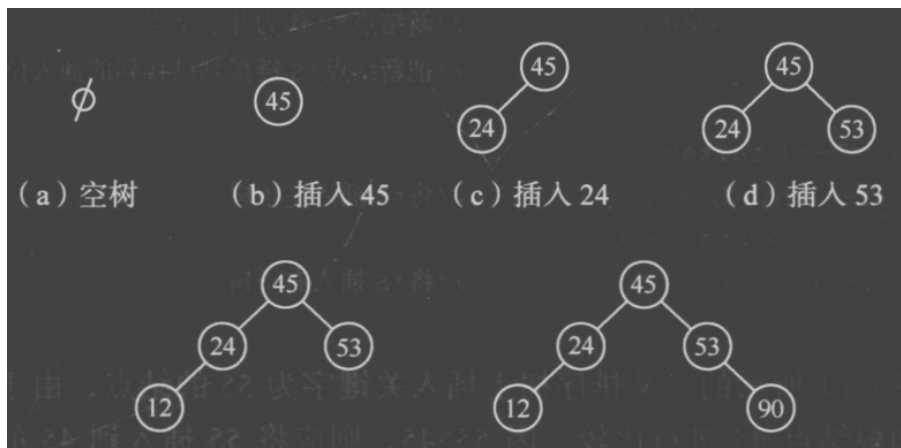
## 测试代码

```
#include "BSTree.h"

int main(void) {
    BSTree T = NULL;
    Creat_BST(&T);
    // Insert_BST(&T, 66);
    InOrder_BST(T);
    system("pause");
    return 0;
}

// 45 24 53 12 90 0
```

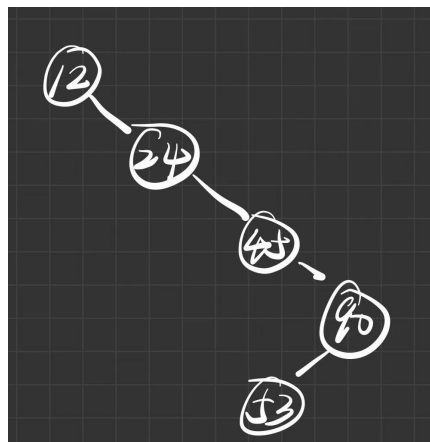
如下图



注意：

不同序列产生的二叉排序树的形态不一样

45 24 53 12 90 (如上图)和 12 24 45 90 53 (如下图)中序遍历的顺序一样，但是形态不一样



已知插入的 $ASL$ 为 $O(\log n)$ ，一共有个节点则需要 $n$ 次循环  
所以创建时间效率为 $O(n \log 2)$

## - BST删除

从BST删除一个节点，不能把以该节点为根的子树都删除，只能删除该节点，并且还要保证删除后的二叉树仍然为BST

BST的删除操作分为三种情况

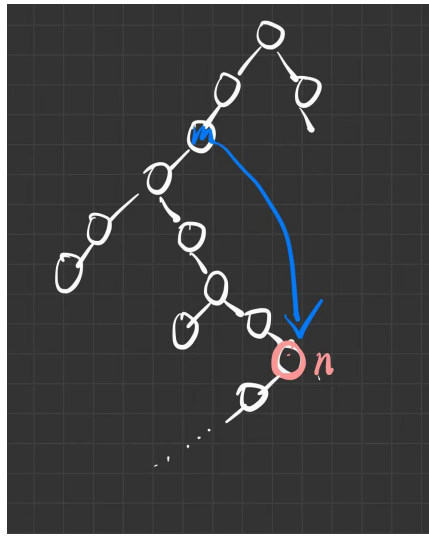
令被删除节点的地址为 $p$ ， $p$ 的双亲结点为 $f$ 且 $p$ 为 $f$ 的左孩子

当 $p$ 为叶子节点时，`f->lchild = NULL;`

当 $p$ 只有一个左子树或右子树时，`f->lchild = p->lchild;` 或 `f->lchild = p->rchild;`

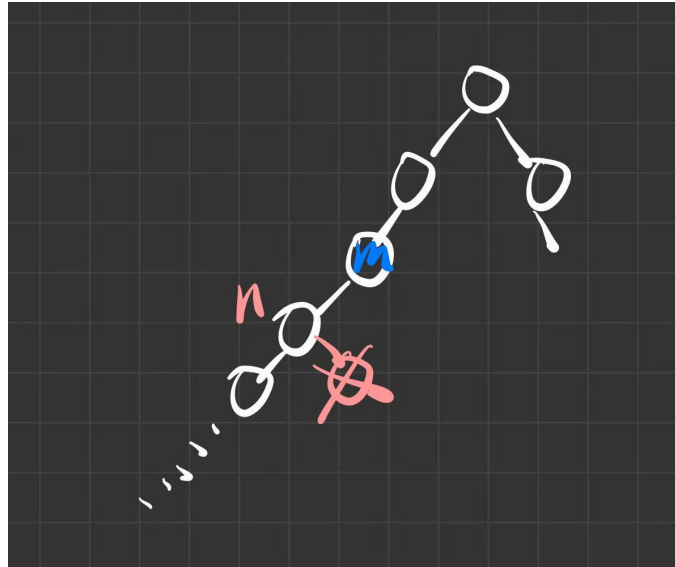
当 $p$ 拥有左右子树时,此种情况较为复杂，需要具体分析

前置知识，若已知一个二叉排序树的节点 $m$ ， $m$ 的直接前驱节点为 $m$ 的左子树上右分支最后一个右孩子为空的节点，如下图



由图可知，**n没有右孩子**(如果n一旦有了有孩子，那么m的直接前驱必然会发生变化)，**但n可以有左孩子**(即使有左孩子也并不影响n是m的直接前驱)

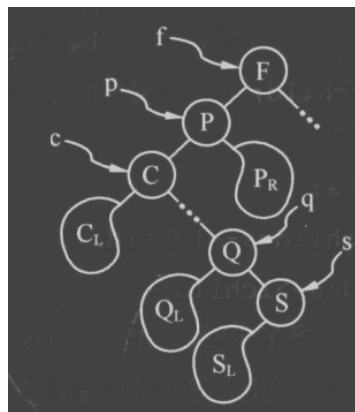
同时也要考虑n的左子树没有右分支的情况，如下图



由图可知，**如果n没有右分支，那么m的直接前驱为n**

利用对称性可知直接后继节点则为：m的**右子树上左分支上最后一个左孩子为空的节点**

考虑下图：



中序遍历所得到的序列为:  $C_L C \dots Q_L Q S_L S P P_R F \dots$

由图可知 P 的直接前驱为 S，如果把序列中 P 删除并且 P 的数据用 S 代替  $C_L C \dots Q_L Q S_L S P_R F \dots$  此时  $S_L$  变成了 Q 的直接前驱为 Q 的右子树上的左分支最后一个左孩子为空的节点为  $S_L$ 。即  $Q \rightarrow$

$\rightarrow \text{right} = S \rightarrow \text{left}$

```
void Delete_BST(BSTree *T, BSTKeyType key) {
    BSTree p = *T;
    BSTree parent = NULL;
    while (p) { //若循环结束则直接return，说明没找key
        if (p->data.key == key)
            break; //退出循环此时p指向要删除节点
        else if (key < p->data.key) {
            parent = p; //通过比较key值定位要删除节点
            p = p->lchild;
        } else {
            parent = p;
            p = p->rchild;
        }
    }

    if (!p) {
        return;
    }
    //当控制来到次行时，说明p指向了要删除的节点
    BSTree pfree;
    BSTree node;
    if (p->lchild && p->rchild) { //第一种情况 p的左右子树不为空
        BSTree prior = p->lchild; // p的直接前驱一定在p的左子树上，所以prior = p的左孩子
        BSTree parent_prior = p; //需要一个节点来定位prior的双亲结点

        while (prior->rchild) { //在右分支上寻找要删除节点p的直接前驱
            parent_prior = prior;
            prior = prior->rchild;
        }

        p->data.key = prior->data.key; //把前驱节点的值赋给要删除节点
        //此时问题转化成了：在保持序列顺序的前提下，如果链接二叉排序树

        if (parent_prior != p) {
            parent_prior->rchild = prior->lchild;
        } else { //此种情况p的直接前驱为p的左孩子，
            parent_prior->lchild = prior->lchild;
        }

        free(prior);
        return;
    } else if (!p->lchild) {
        pfree = p; // pfree用于存放 要删除节点的地址
        node = p->rchild; // node存放需要链接节点的地址
    } else if (!p->rchild) {
        pfree = p;
        node = p->lchild;
    }

    //当控制来到此行时，pfree保存要删除节点的地址，node存放需要连接的节点地址
    if (!parent) { //如果parent域仍然为空，说明要删除的节点为根节点
        *T = node;
    }
}
```

```

    } else if (parent->rchild == p) {
        parent->rchild = node;
    } else {
        parent->lchild = node;
    }

    free(pfree);
}

```

测试代码

```

#include "BSTree.h"

int main(void) {
    BSTree T = NULL;
    Creat_BST(&T);
    Delete_BST(&T, 53);
    Delete_BST(&T, 12);
    InOrder_BST(T);
    system("pause");
    return 0;
}

// 45 24 53 12 90 0

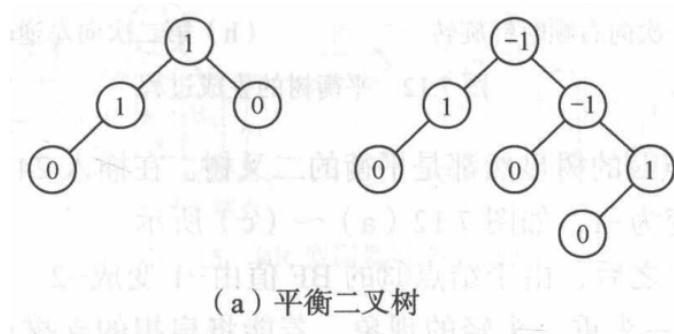
```

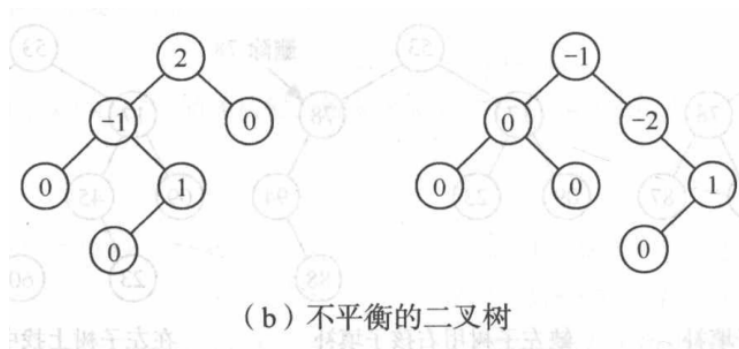
## • AVL树(Balance Binary Tree)

平衡二叉树排序树(AVL树):

需要满足如下的三个性质:

- (1)有一个树 $T$ , 令 $T$ 的左子树的高度为 $LH$ , 右子树高度为 $RH$   
平衡因子( $Balance\ Factor$ ) =  $|LH - RH| \leq 1$
- (2) $T$ 的左右子树也为平衡二叉排序树
- (3)如果插入节点使得 $Balance\ Factor = \pm 2$   
则必须旋转 $BF$ 为2的节点



**定理1:**

若一个AVL树 $T$ 在添加一个节点 $n$ 后,  $T_{BF} = \pm 2$ , 则 $n$ 的双亲结点 $A$ 的 $BF$ 不能为0(叶子节点除外)

*Proof:*

令一颗AVL树 $T$ 在添加一个节点 $n$ 后 $T_{BF} = \pm 2$ , 且 $n$ 的双亲节点 $A$ 的 $BF = 0$

若删去 $n$ 节点, 则 $A$ 的高度并未发生变化, 且 $T_{BF}$ 未发生变化

即 $T_{BF} = 2$ , 说明 $T$ 在添加 $n$ 节点之前不是AVL树

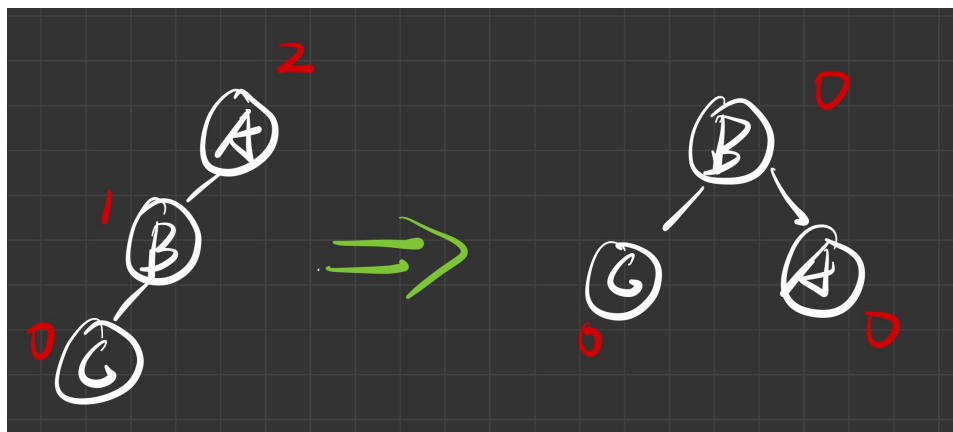
*Contradiction*( $T$ 不是一颗AVL树)

**LL型旋转(右旋转)**

情况(1)

$A_{BF} = 2$ 且 $B_{BF} = 1$ , 以 $A$ 的左孩子 $B$ 为中心, 向右旋转

如下图 $C$ 为插入节点

**定理2**

即将进行LL旋转的树 $A_{BF} = 2$ ,  $B_{BF} = 1$ 且 $C_{BF} = 0$

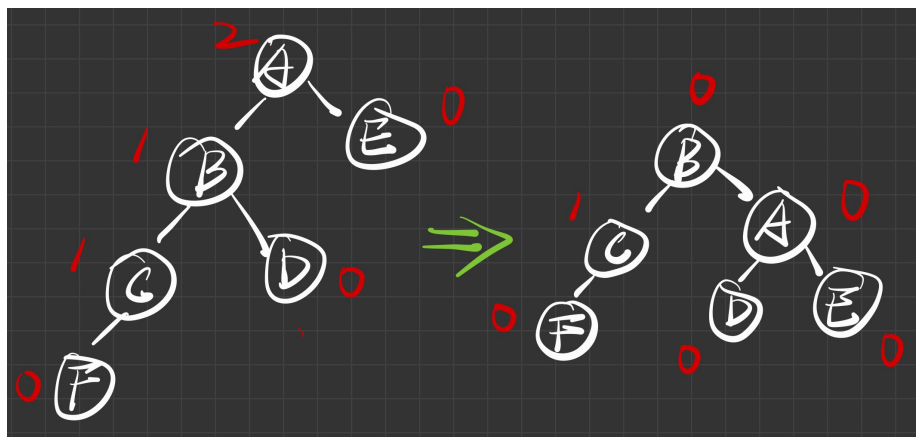
则旋转后 $B_{BF} = 0$ ,  $C_{BF} = 0$ ,  $A_{BF} = 0$

情况(2)

$A_{BF} = 2$ 且 $B_{BF} = 1$ , 以 $A$ 的左孩子 $B$ 为中心向右旋转, 此时 $C$ 成为 $B$ 的左孩子,  $A$ 成为 $B$ 的右孩子

则 $D$ 节点脱落, 使 $D$ 成为 $A$ 的右孩子

如下图 $F$ 为插入节点

**定理3**

即将进行LL旋转的树  $A_{BF} = 2$ ,  $B_{BF} = 1$  且  $C_{BF} = 1$

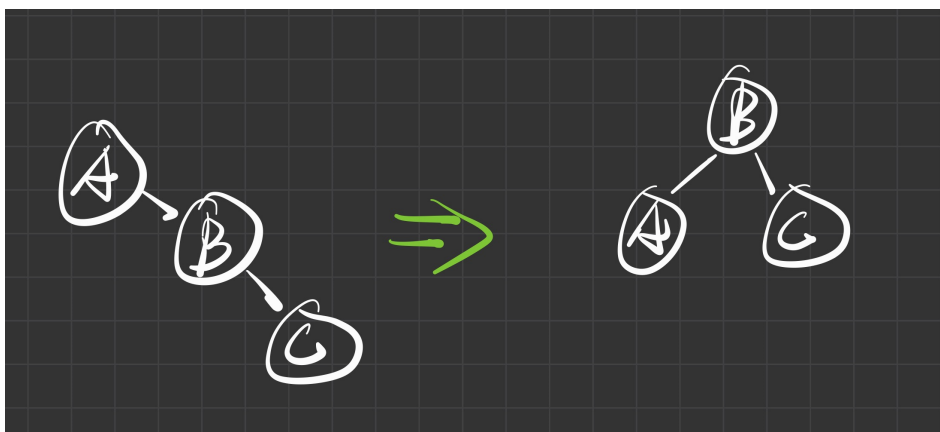
则旋转后  $B_{BF} = 0$ ,  $C_{BF} = 1$ ,  $A_{BF} = 0$

**RR旋转(左旋转)**

情况(1)

$A_{BF} = -2$  且  $B_{BF} = -1$ , 以A的右孩子B为中心, 向左旋转

如下图C为插入节点

**定理4**

即将进行RR旋转的树  $A_{BF} = -2$ ,  $B_{BF} = -1$  且  $C_{BF} = 0$

则旋转后  $B_{BF} = 0$ ,  $C_{BF} = 0$ ,  $A_{BF} = 0$

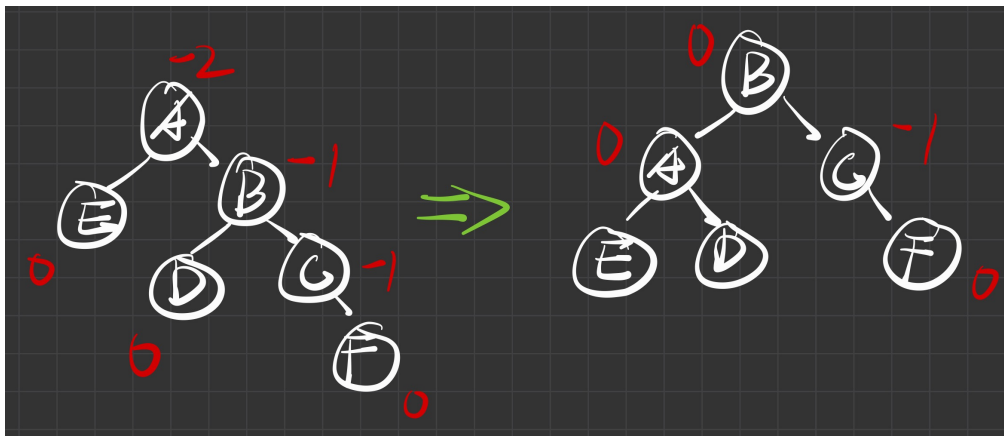
情况(2)

$A_{BF} = -2$  且  $B_{BF} = -1$ , 以A的右孩子B为中心向左旋转, 此时C成为B的右孩子, A称成为B的左孩子

则D节点脱落, 使D成为A的右孩子

如下图F为插入节点



**定理5**

即将进行RR旋转的树  $A_{BF} = -2$ ,  $B_{BF} = -1$  且  $C_{BF} = -1$

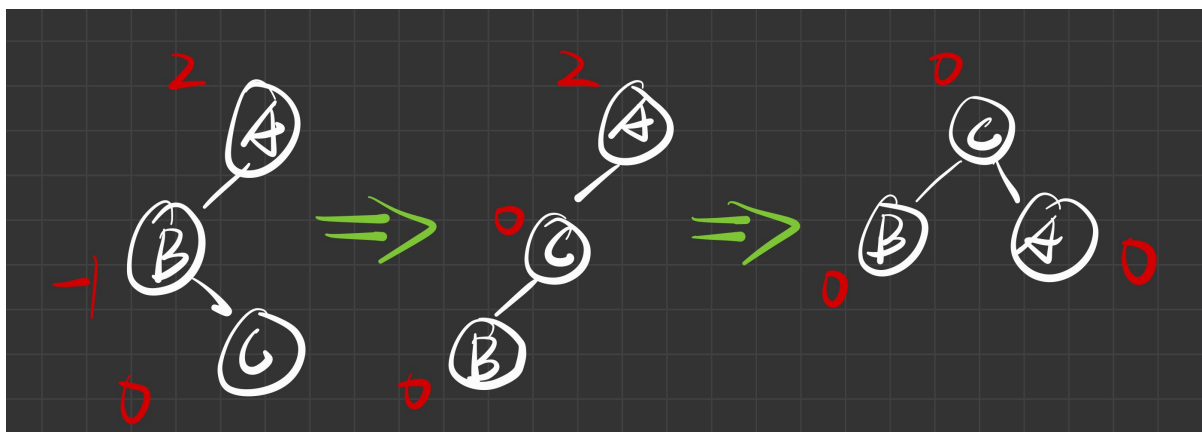
则旋转后  $B_{BF} = 0$ ,  $C_{BF} = -1$ ,  $A_{BF} = 0$

**LR旋转(左右旋转)**

情况(1)

即将进行旋转的树  $A_{BF} = 2$  且  $B_{BF} = -1$ , 则RR旋转B, 再LL旋转A

如下图, C为插入节点

**定理6**

即将进行LR旋转的树  $A_{BF} = 2$ ,  $B_{BF} = -1$  且  $C_{BF} = 0$

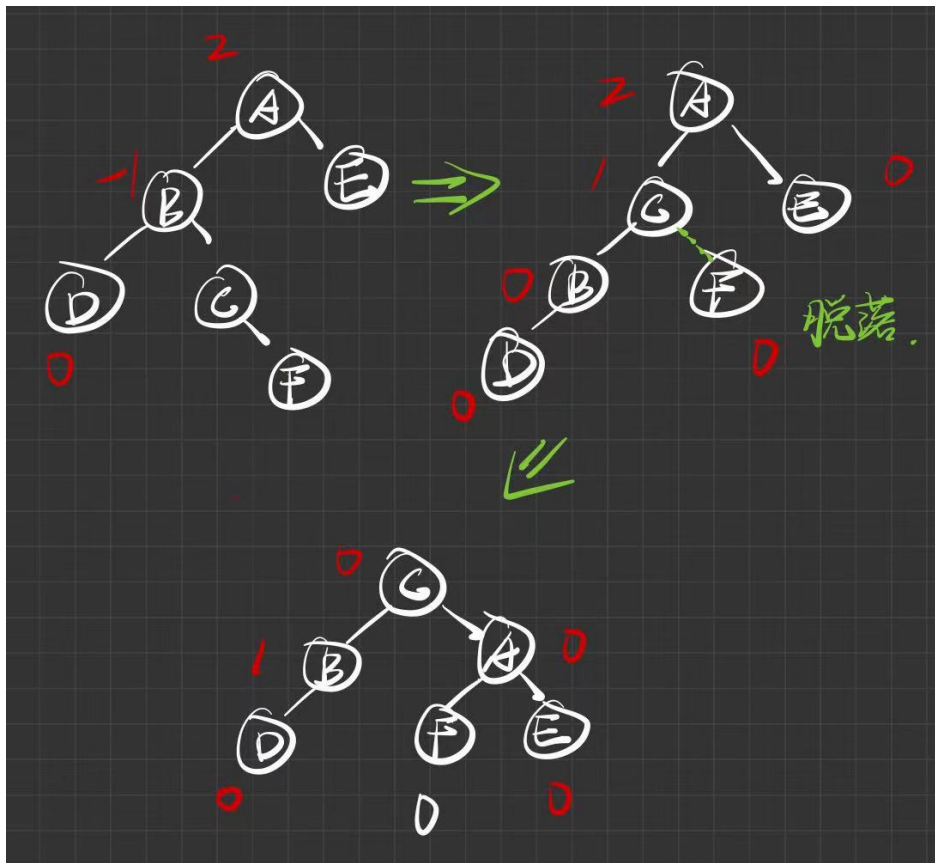
则LR旋转后  $B_{BF} = 0$ ,  $C_{BF} = 0$ ,  $A_{BF} = 0$

情况(2)

节点C有右孩子F, 即  $C_{BF} = -1$

即将进行旋转的树  $A_{BF} = 2$  且  $B_{BF} = -1$ , 则RR旋转B(F节点脱落), 再LL旋转A, 使F成为A的左孩子

如下图, F为插入节点

**定理7**

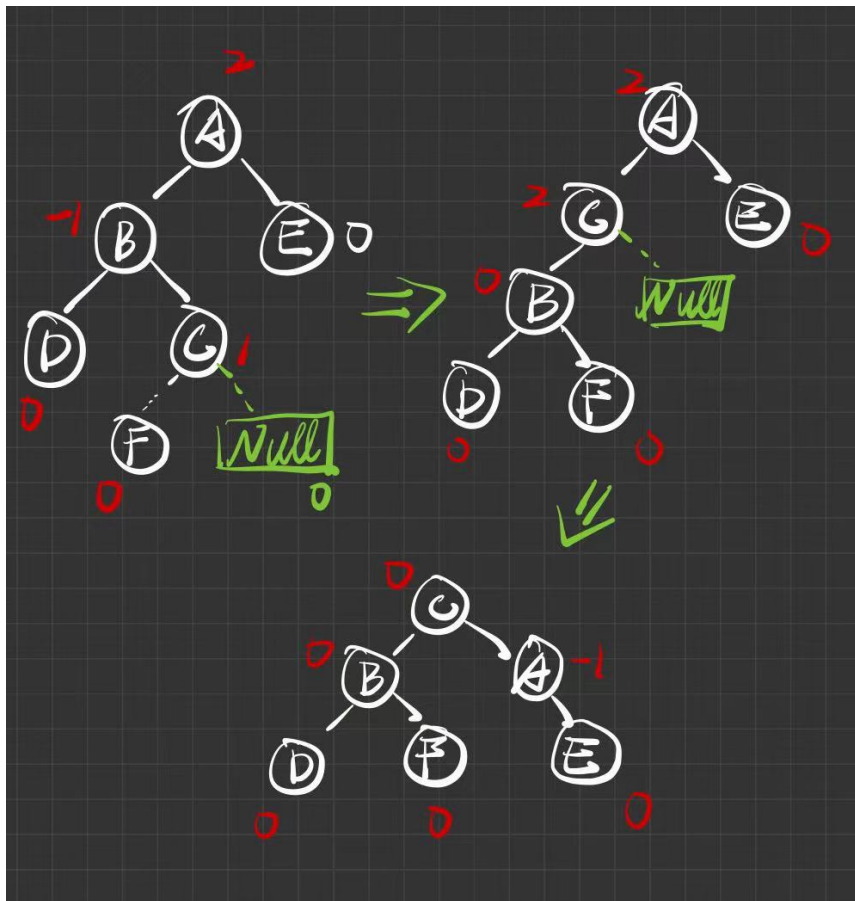
即将进行 $LR$ 旋转的树 $A_{BF} = 2$ ,  $B_{BF} = -1$ 且 $C_{BF} = -1$

则 $LR$ 旋转后 $B_{BF} = 1$ ,  $C_{BF} = 0$ ,  $A_{BF} = 0$

情况(3)

节点 $C$ 有左孩子 $F$ , 即 $C_{BF} = 1$

即将进行 $LR$ 旋转的树 $A_{BF} = 2$ 且 $B_{BF} = -1$ , 则 $RR$ 旋转 $B$ ( $F$ 节点脱落), 再 $LL$ 旋转 $A$ , 使 $F$ 成为 $A$ 的有孩子  
如下图,  $F$ 为插入节点



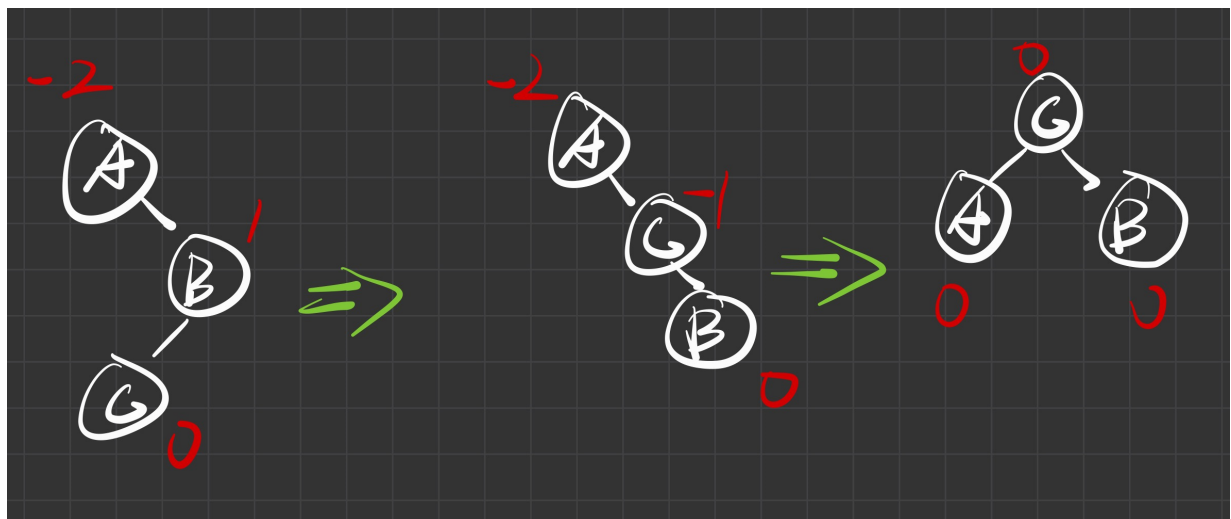
#### 定理8

即将进行 $LR$ 旋转的树 $A_{BF} = 2$ ,  $B_{BF} = -1$ 且 $C_{BF} = 1$   
 则 $LR$ 旋转后 $B_{BF} = 0$ ,  $C_{BF} = 0$ ,  $A_{BF} = -1$

#### - RL旋转(右左旋转)

情况(1)

即将进行 $RL$ 旋转的树 $A_{BF} = -2$ 且 $B_{BF} = 1$ , 则 $LL$ 旋转 $B$ , 再 $RR$ 旋转 $A$   
 如下图,  $C$ 为插入节点



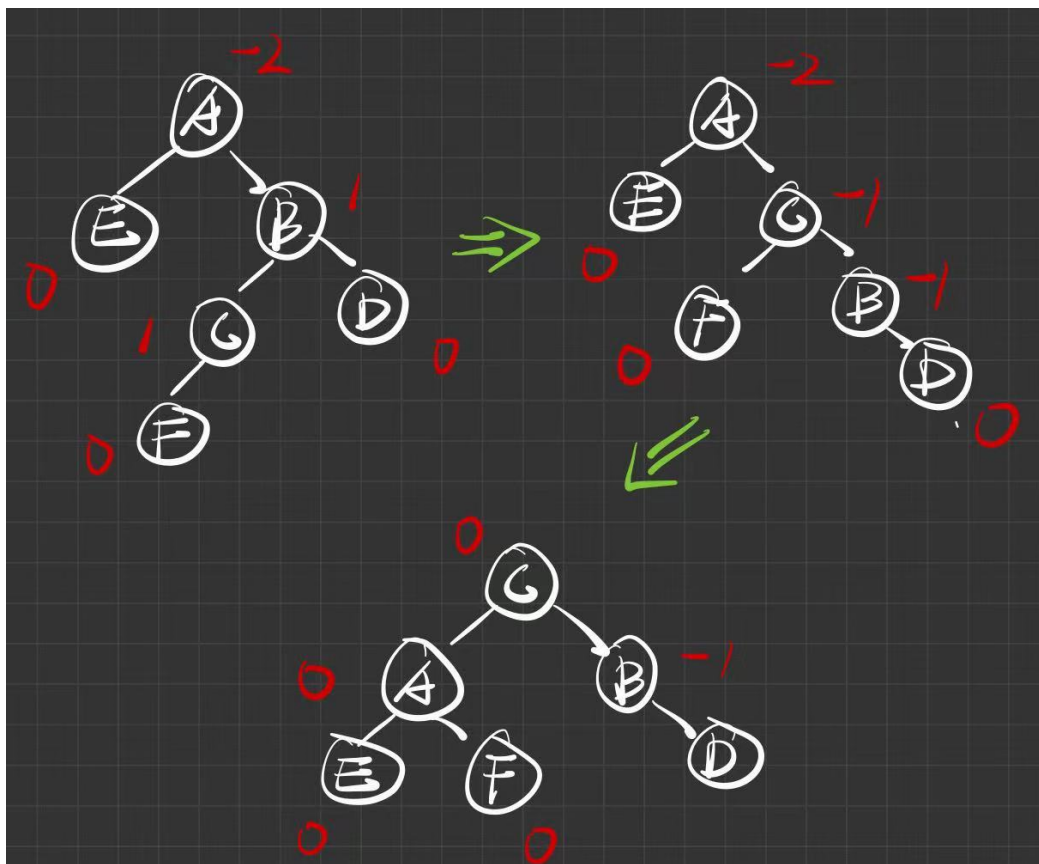
#### 定理9

即将进行 $RL$ 旋转的树 $A_{BF} = -2$ ,  $B_{BF} = 1$ 且 $C_{BF} = 0$   
 则 $RL$ 旋转后 $B_{BF} = 0$ ,  $C_{BF} = 0$ ,  $A_{BF} = 0$

## 情况(2)

节点 $C$ 有左孩子 $F$ , 即 $C_{BF} = 1$

即将进行 $RL$ 旋转的树 $A_{BF} = -2$ 且 $B_{BF} = 1$ , 则 $LL$ 旋转 $B$ ( $F$ 节点脱落), 再 $RR$ 旋转 $A$ , 使 $F$ 成为 $A$ 的左孩子  
如下图,  $F$ 为插入节点



## 定理10

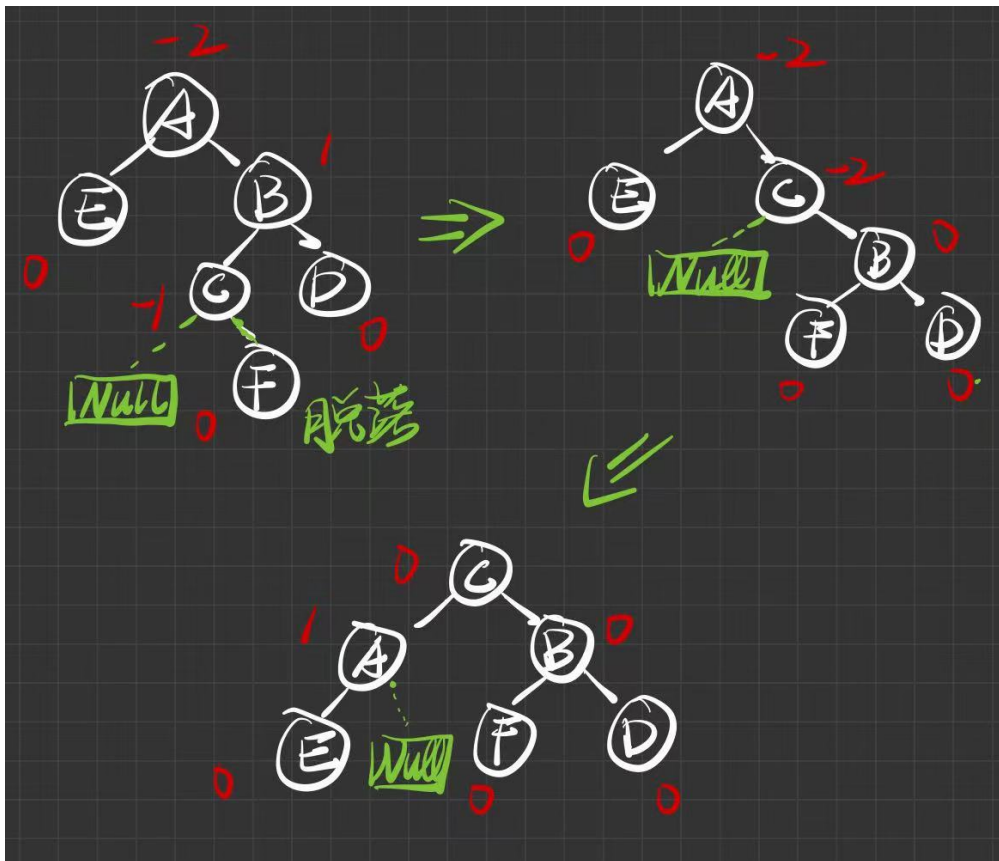
即将进行 $RL$ 旋转的树 $A_{BF} = -2$ ,  $B_{BF} = 1$ 且 $C_{BF} = 1$

则 $LR$ 旋转后 $B_{BF} = -1$ ,  $C_{BF} = 0$ ,  $A_{BF} = 0$

## 情况(2)

节点 $C$ 有右孩子 $F$ , 即 $C_{BF} = -1$

即将进行 $RL$ 旋转的树 $A_{BF} = -2$ 且 $B_{BF} = 1$ , 则 $LL$ 旋转 $B$ ( $F$ 节点脱落), 再 $RR$ 旋转 $A$ , 使 $F$ 成为 $A$ 的左孩子  
如下图,  $F$ 为插入节点



### 定理11

即将进行 $RL$ 旋转的树 $A_{BF} = -2$ ,  $B_{BF} = 1$ 且 $C_{BF} = -1$   
 则 $LR$ 旋转后 $B_{BF} = 0$ ,  $C_{BF} = 0$ ,  $A_{BF} = 1$

### 总结

如果一个 $AVL$ 树 $T$ 参入一个节点使得 $T$ 的某个子树的 $BF = \pm 2$ , 则需要对该子树进行旋转

令这颗子树为 $A$ ,  $A_{BF} = \pm 2$

- (1)如果 $A_{BF} = 2$ , 且 $A$ 的左子树 $BF$ 为 $1$ , 则对 $A$ 进行 $LL$ 旋转
- (2)如果 $A_{BF} = 2$ , 且 $A$ 的左子树 $BF$ 为 $-1$ , 则对 $A$ 进行 $LR$ 旋转
- (3)如果 $A_{BF} = -2$ , 且 $A$ 的左子树 $BF$ 为 $-1$ , 则对 $A$ 进行 $RR$ 旋转
- (4)如果 $A_{BF} = -2$ , 且 $A$ 的左子树 $BF$ 为 $1$ , 则对 $A$ 进行 $RL$ 旋转

### 代码实现

数据类型:

```
#define LH 1 //平衡因子1
#define EH 0 //平衡因子0
#define RH -1 //平衡因子-1

typedef int AVLElemtype;
typedef struct __AVLNode {
    AVLElemtype key;
    int bf; // balance factor
    __AVLNode *lchild, *rchild;
} AVLNode, *AVLTree;
```

## 左旋转和右旋转(参照LL, RR旋转)

左旋转树 $T$ 代表着：以 $T$ 的右孩子为中心，向左旋转

右旋转树 $T$ 代表着：以 $T$ 的左孩子为中心，向右旋转

```
#include "AVLTree.h"

void LeftRotate(AVLTree *T) {
    AVLTree Rchild = (*T)->rchild; // Rchild代表T的右孩子
    (*T)->rchild = Rchild->lchild; //参照RR旋转
    Rchild->lchild = *T;           //旋转后T成为Rchild的左孩子(定理3)
    *T = Rchild;
    /// 上行代码不可省略，如果省略，则只改动了节点间的关系，而未改变指针之间的关系
}

void RightRotate(AVLTree *T) {
    AVLTree Lchild = (*T)->lchild;
    (*T)->lchild = Lchild->rchild;
    Lchild->rchild = *T;
    *T = Lchild;
}
```

## - 左平衡和右平衡

根据[总结](#)可知，大体上可以分成左平衡(LL,LR)和右平衡(RR,RL)

### 左平衡

```
void LeftBalance(AVLTree *T) {
    AVLTree L = (*T)->lchild; // L->bf绝对不可能为EH(定理1)
    AVLTree Lr;                // T的左孩子的右孩子
    switch (L->bf) {
        // LL 旋转
        case LH:
            /// 定理2
            (*T)->bf = L->bf = EH;
            /// 此行的上下两行不可对调
            RightRotate(T);
            break;
        case RH:
            // LR旋转
            Lr = L->rchild; //
            switch (Lr->bf) {
                case LH:
                    //定理8
                    (*T)->bf = RH;
                    L->bf = EH;
                    break;
                case EH:
                    //定理6
                    (*T)->bf = L->bf = EH;
                    break;
            }
    }
}
```

```

    case RH:
        //定理7
        (*T)->bf = EH;
        L->bf = LH;
        break;
    }
    //根据定理6, 7, 8可知, 旋转后Lr的BF必定为0
    Lr->bf = EH;
    LeftRotate(&(*T)->lchild);
    RightRotate(T);
    break;
}
}

```

## 右平衡

```

// 和左平衡同理
void RightBalance(AVLTree *T) {
    AVLTree R = (*T)->rchild;
    AVLTree Rl;
    switch (R->bf) {
    case RH:
        (*T)->bf = R->bf = EH;
        LeftRotate(T);
        break;
    case LH: {
        Rl = R->lchild; //
        switch (Rl->bf) {
        case LH:
            (*T)->bf = EH;
            R->bf = RH;
            break;
        case EH:
            (*T)->bf = R->bf = EH;
            break;
        case RH:
            (*T)->bf = LH;
            R->bf = EH;
            break;
        }
        Rl->bf = EH;
        RightRotate(&(*T)->rchild);
        LeftRotate(T);
    }
    }
}
}

```

## 插入节点和及时平衡

```

    /// 全局变量taller记录高度是否发生变化，如果未发生变化为false，发生则true
    bool __taller = false;
    void Insert_AVL(AVLTree *T, AVLElement key, bool *taller) {
        // T为要插入节点的双亲节点，key为要插入数据的值
        //若T为空，则创建一个节点，并初始化
        if (!(*T)) {
            *T = (AVLTree)malloc(sizeof(AVLNode));
            (*T)->lchild = (*T)->rchild = NULL;
            (*T)->bf = EH;
            (*T)->key = key;
            *taller = true;
        }
        if (key < (*T)->key) {
            //如果插入值小于T的key值，则递归T的左子树，直到找到一个NULL节点
            Insert_AVL(&(*T)->lchild, key, taller);
            //判断树是否变高了
            if (*taller) {
                //以T为根插入节点，则T的bf发生变化
                switch ((*T)->bf) {
                    case LH:
                        // T的bf == -1，向T的左孩子插入节点则T的bf = 2，需要左平衡
                        LeftBalance(T);
                        *taller = false;
                        //经过平衡后taller = false，因为T经过左调整后，变得平衡了
                        break;
                    case EH:
                        // 如果T的bf == 0，向T的左孩子插入节点，则T的bf = 1
                        (*T)->bf = LH;
                        //此时T的高度发生变化
                        *taller = true;
                        break;
                    case RH:
                        // 如果T的bf == -1，向T的左孩子插入节点，则T的bf = 0
                        (*T)->bf = EH;
                        //高度未发生变化
                        *taller = false;
                        break;
                }
            }
        }
        else {
            //和上面同理
            Insert_AVL(&(*T)->rchild, key, taller);

            if (*taller) {
                switch ((*T)->bf) {
                    case LH:
                        (*T)->bf = EH;
                        *taller = false;
                        break;
                    case EH:
                        (*T)->bf = RH;
                        *taller = true;
                        break;
                    case RH:
                        RightBalance(T);

```



```

        *taller = false;
        break;
    }
}
}
}

```

## - 测试代码

```

#include "AVLTree.h"

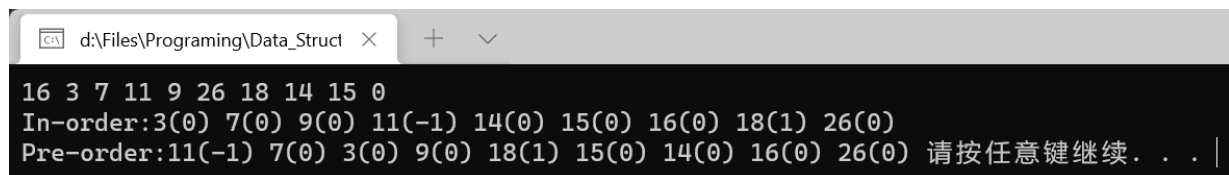
void Creat_AVL(AVLTree *T);

extern bool __taller;
int main(void) {
    AVLTree T = NULL;
    Creat_AVL(&T);
    printf("In-order:");
    Traverse_Inorder(T);
    printf("\nPre-order:");
    Traverse_Preorder(T);
    system("pause");
    return 0;
}
// 输入数据: 16 3 7 11 9 26 18 14 15 0

void Creat_AVL(AVLTree *T) {
    *T = NULL;
    AVLElement key;
    scanf("%d", &key);
    // 0 代表输入结束
    while (key != 0) {
        Insert_AVL(T, key, &__taller);
        scanf("%d", &key);
    }
}

```

output:



```

16 3 7 11 9 26 18 14 15 0
In-order:3(0) 7(0) 9(0) 11(-1) 14(0) 15(0) 16(0) 18(1) 26(0)
Pre-order:11(-1) 7(0) 3(0) 9(0) 18(1) 15(0) 14(0) 16(0) 26(0) 请按任意键继续...

```

## • 哈希表(Hash Table)

记录存储位置与关键字之间存在对应关系

用  $LOC(i) = Hash(key_i)$  表示

优点: 查找效率高

缺点：空间效率低

冲突：通过Hash函数，不同的关键字映射到同一个地址上

## • hash函数的构造方法

需要考虑的因素：

- 执行速度(计算时间)
- 关键字长度
- Hash Table的大小
- 关键字的分布情况
- 朝朝频率

主要构造方法：

- 直接定址法
- 数字分析法
- 平方取中法
- 折叠法
- 除留余数法
- 随机数法

直接定址法：

$$Hash(key) = a \cdot key + b \text{ with } a, b \text{ is constant}$$

优点：以关键字key的某个线性函数值为散列地址，不会产生冲突

缺点：要占用连续地址空间，空间效率低

例

$$\{100, 300, 500, 700, 800, 900\}$$

$$\text{散列函数 } Hash(key) = key/100$$

## - 处理冲突的方法

除留余数法：

$$H_i = (Hash(key) + d_i) \bmod m \quad (d_i \text{ 为增量序列}, m \leq \text{表长}, \text{且 } m \text{ 是个 prime 质数})$$

开放定址法：

基本思想：有冲突时就去寻找下一个空的散列地址，只要表足够大，总能找到空的地址

常用方法：

线性探测法： $d_i$ 为 $1, 2, 3, \dots, m-1$

二次探测法： $d_i$ 为 $1^2, -1^2, 2^2, -2^2, \dots, q^2$

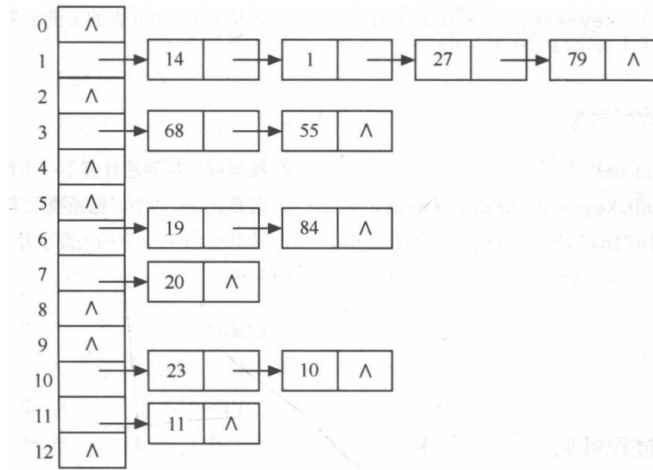
伪随机数探测法： $d_i$ 为伪随机数数列

### 链地址法：

基本思想，相同散列地址的记录链成一单链表

例：

已知一组关键字为 $\{19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79\}$ ，令 $Hash(key) = key \bmod 13$



链地址法的优点：

- 非同义词不会冲突，无聚集现象
- 链表上空间动态申请，更适用于表长不确定的情况

## - 除留余数法构造Hash Table

类型定义：

```

#define __m 11    // __m是表长
#define __n 9     // 元素个数
#define NULLKEY 0 // 0意味着表空
typedef int HashElemType;
typedef char HashOther;
typedef struct __HashTable {
    HashElemType key;
    HashOther other;
} HashTable[__m];

```

初始化并构造：

```

void InitHashTable(HashTable hash) {
    printf("Please input %d integer(s):", __n);
    int key;
    // 初始化
    memset(hash, 0, sizeof(HashTable));
}

```

```

for (int i = 1; i < __n + 1; i++) {
    scanf(" %d", &key);
    int m_i = Hash(key);

    if (hash[m_i].key == NULLKEY) {
        hash[m_i].key = key;
    } else {
        for (int j = 1; j < __m; j++) {
            int m_j = Hash(key + j);

            if (hash[m_j].key == NULLKEY) {
                hash[m_j].key = key;
                break;
            }
        }
    }
}
}
}
}

```

搜索:

```

int Hash_Search(HashTable hash, HashElemType key) {
    int m_i = Hash(key);

    if (hash[m_i].key == NULLKEY)
        return -1;
    else if (hash[m_i].key == key)
        return m_i;
    else {
        for (int i = 1; i < __m; i++) {
            int m_j = Hash(key + i);
            if (hash[m_j].key == NULLKEY)
                return -1;
            else if (hash[m_j].key == key)
                return m_j;
        }
        return -1;
    }
}
}

```

## - 查找效率分析

Hash Table的ASL取决于:

- 散列函数
- 处理冲突的方法
- 散列表的装填因子 $\alpha(\frac{\text{表中元素个数}}{\text{表长}})$   $\alpha$ 越接近1说明发生冲突的可能性越大

所以说Hash Table的查找效率 既不是 $O(1)$ , 也不是 $O(n)$

$$ASL \approx 1 + \frac{\alpha}{2} (\text{拉链法})$$

$$ASL \approx \frac{1}{2} \left( 1 + \frac{1}{1 - \alpha} \right) (\text{线性探测法})$$

$$ASL \approx -\frac{1}{\alpha} \ln(1 - \alpha) (\text{随机探测法})$$

## 几点结论

- 散列表技术具有很好的平均性能，优于一些传统技术
- 链地址法优于开地址法
- 除留余数法作散列函数优于其他类型函数

## 查找总结

顺序查找，折半查找，分块查找比较

|       | 顺序查找               | 折半查找             | 分块查找                     |
|-------|--------------------|------------------|--------------------------|
| 时间复杂度 | $O(n)$             | $O(\log^n)$      | 与确定所在块的查找方法有关            |
| 特点    | 算法简单，对结构无要求，效率底    | 对表结构有要求，效率高      | 对结构有一定要求，效率介于顺序查找和折半查找之间 |
| 适用情况  | 任何结构的线性表，不经常做插入和删除 | 有序的顺序表，不经常做插入和删除 | 块间有序，块内无序的循环表，经常做插入和删除   |

折半查找和二叉排序树比较

|       | 折半查找                 | 二叉排序树                    |
|-------|----------------------|--------------------------|
| 时间复杂度 | $O(\log^n)$          | $O(\log^n)$              |
| 特点    | 有序的顺序表，插入和删除需要移动大量元素 | 用二叉链表，插入和删除无需移动元素，只需修改指针 |
| 适用情况  | 不经常插入删除              | 经常插入和删除                  |

哈希表:开地址法和链地址法比较

|       | 开地址法            | 链地址法             |
|-------|-----------------|------------------|
| 空间    | 无指针域，存储效率高      | 附加指针域            |
| 时间复杂度 | 有二次聚集现象，查找效率低   | 无二次聚集现象，查找效率高    |
| 插入删除  | 不易实现            | 易于实现             |
| 适用情况  | 表的大小固定，适用于表长无变化 | 节点动态生成，适用于表长经常变化 |

## 排序

排序方法的分类：

- 按数据存储介质：内部排序和外部排序
  - 内部排序：数据量不大，数据在内存，无需内外存交换数据
  - 外部排序：数据量较大，数据在外存(文件排序)
  - 外部排序时，要将数据分批调入内存来排序，中间结果还要及时存入外存
- 按比较器个数：串行排序和并行排序
  - 串行排序：单处理机(同一时刻比较一对元素)
  - 并行排序：多处理机(同一时刻比较多对元素)
- 按主要操作：比较排序和基数排序
  - 比较排序：用比较的方法
  - 基数排序：仅仅根据数据本身的取值
- 按辅助空间排序：原地排序和非原地排序
  - 原地排序：辅助空间为 $O(1)$ 的排序
  - 非原地排序：辅助空间大于 $O(1)$ 的排序
- 按稳定性：稳定排序和非稳定排序
  - 稳定排序：任何数值相等的元素，排序以后相对次序不变
  - 排序方法是否稳定，并不能衡量一个排序算法的优劣
- 按自然性：自然排序和非自然排序
  - 自然排序：输入数据越有序，排序的速度越快

排序类型定义：

```
#define SORTSIZE 20
typedef int SortType;
typedef char SortOther;
typedef struct __RecordType {
    SortType key;
    SortOther other;
} RecordType;
typedef struct __RecordList {
    RecordType r[SORTSIZE + 1]; //数组中的0号位置作guard
    int length;
} RecordList;
```

## • 直接插入排序

若有 $n$ 个元素需要进行排序，令 $i_1$ 到 $i_k$ 为有序表(非递减)， $i_{k+1}$ 元素与该有序表比较，插入到适当位置

$i_1$ 为有序表(只有一个元素)， $i_2$ 与该有序表比较，进行插入，此时有序表变成 $i_1$ 到 $i_2$

依此进行插入 $i_3, i_4, \dots, i_n$

此时 $i_1$ 到 $i_n$ 为非递减序列

代码实现：

```
void InsertSort(RecordList *List) {
    for (int i = 2; i <= List->length; i++) {
        // 因为有guard所以, i = 2, 而不是 i = 1;
        if (List->r[i].key < List->r[i - 1].key) {
            // 判断下标为i的元素是否大于i-1, 如果大于则不需要进行排序
            // 如果小于则需要进行如下排序
            List->r[0].key = List->r[i].key; //设置guard
            List->r[i] = List->r[i - 1];    // 向后移动

            int j;
            for (j = i - 2; List->r[0].key < List->r[j].key; j--) {
                List->r[j + 1] = List->r[j];
            }
            // 当上面循环结束后, 下标为j的元素此时小于guard, 则向j的直接后继(j+1)插入
            List->r[j + 1] = List->r[0];
        }
    }
}
```

测试代码：

```
#include "InsertSort.h"

void input(RecordList *RL);

int main(void) {
    RecordList RL;
    RL.length = 9;
    input(&RL);
    InsertSort(&RL);

    for (int i = 1; i <= RL.length; i++) {
        printf("%d ", RL.r[i].key);
    }

    system("pause");
    return 0;
}
// 输入数据: 47 7 29 11 16 92 22 8 3

void input(RecordList *RL) {
    RecordType *p = RL->r + 1;
    for (int i = 1; i <= RL->length; i++) {
```

```

SortType key;
scanf(" %d", &key);
p->key = key;
p++;
}
p = NULL;
}

```

性能分析:

最坏情况 :

$$\text{比较的次数} \sum_{i=2}^n i = \frac{(n+2)(n-1)}{2}$$

$$\text{移动的次数} \sum_{i=2}^n (i+1) = \frac{(n+4)(n-1)}{2}$$

平均情况

$$\text{比较次数} \sum_{i=1}^{n-1} \frac{i+1}{2} = \frac{1}{4}(n+2)(n-1)$$

$$\text{移动次数} \sum_{i=1}^{n-1} \left( \frac{i+1}{2} + 1 \right) = \frac{1}{4}(n+6)(n-1)$$

最坏情况下 :  $O(n^2)$

平均 :  $O(n^2)$

## • 折半插入排序(Binary Insert Sort)

在直接插入排序的基础上，对已经排序好的有序表进行折半操作，随着折半的进行，right+1就是插入的位置

```

void InserSotr_Binary(RecordList *List) {
    for (int i = 2; i <= List->length; i++) {
        List->r[0] = List->r[i];
        int left = 1;
        int right = i - 1;

        while (left <= right) {
            int mid = (left + right) / 2;
            if (List->r[0].key > List->r[mid].key) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }

        for (int j = i - 1; j >= right + 1; j--) {
            List->r[j + 1] = List->r[j];
        }

        List->r[right + 1] = List->r[0];
    }
}

```



```

    }
}

```

折半插入性能:

时间复杂度： $O(n^2)$

空间复杂度： $O(1)$

是一种稳定排序

### 直接插入和折半插入的比较

- 折半插入的比较次数和待排序序列的初始排列无关，仅依赖序列元素个数
- 折半插入减少了比较次数，但是没有减少移动次数
- 折半插入平均性能优于直接插入排序
- 直接插入在基本有序时，效率更高

## • 希尔排序(Shell's Sort)

希尔排序思路：

增量序列 $\{D_K\}$ ： $D_M > D_{M-1} > \dots > D_1 = 1$ , 并且*relatively prime*

对每个 $D_K$ 进行间隔插入排序

例如 $\{D_K\} = 5, 3, 1$

则依此对将要排序的序列进行间隔为5, 3, 1的直接插入排序

希尔排序特点:

- 移动位置较大，跳跃式地接近排序后的最终位置
- 最后一次只需要少量移动
- 增量序列必须是递减的，最后一个必须是1
- 增量序列必须是互质的

代码实现:

```

void ShellInsert(RecordList *L, int dk) {
    for (int i = 1 + dk; i <= L->length; i++) {
        if (L->r[i].key < L->r[i - dk].key) {
            L->r[0] = L->r[i];
            L->r[i] = L->r[i - dk];
            int j;
            for (j = i - (2 * dk); j > 0 && L->r[0].key < L->r[j].key; j -= dk) {
                L->r[j + dk] = L->r[j];
            }
            L->r[j + dk] = L->r[0];
        }
    }
}

void ShellSort(RecordList *L, int *dt, int t) {
    for (int i = 0; i < t; i++) {
        ShellInsert(L, dt[i]);
    }
}

```

}

测试代码:

```

#include "InsertSort.h"

void input(RecordList *RL);

int main(void) {
    RecordList RL;
    RL.length = 10;
    int dt[3] = {5, 3, 1};
    input(&RL);
    ShellSort(&RL, dt, 3);

    for (int i = 1; i <= RL.length; i++) {
        printf("%d ", RL.r[i].key);
    }

    system("pause");
    return 0;
}
// 输入数据: 49 38 65 97 76 13 27 49 55 4

void input(RecordList *RL) {
    RecordType *p = RL->r + 1;
    for (int i = 1; i <= RL->length; i++) {
        SortType key;
        scanf(" %d", &key);
        p->key = key;
        p++;
    }
    p = NULL;
}

```

效率分析

*Hibbard*增量序列  $D_K = 2^{k-1}$ , 相邻元素互质

最坏情况:  $O(n^{\frac{3}{2}})$

猜想:  $AVL = O(n^{\frac{5}{4}})$

*Sedgewick*增量序列

$\{1, 5, 19, 41, 109 \dots\}$

$9 \times 4^i - 9 \times 2^i + 1$  或  $4^i - 3 \times 2^i + 1$

猜想:

最坏情况:  $O(n^{\frac{4}{3}})$

$ASL = O(n^{\frac{7}{6}})$

## • 交换排序

### - 冒泡排序

```
void BubbleSort(RecordList *L) {
    bool flag = true;
    for (int i = 1; i <= L->length - 1 && flag; i++) {
        // 注意此行flag的位置,不要写在j循环里面
        flag = false;
        for (int j = 1; j <= L->length - i; j++) {
            if (L->r[j].key > L->r[j + 1].key) {
                // 只要发生一次交换, flag就为true
                flag = true;
                RecordType temp = L->r[j];
                L->r[j] = L->r[j + 1];
                L->r[j + 1] = temp;
            }
        }
    }
}
```

冒泡排序算法效率分析:

最好情况(正序)

比较次数:  $n - 1$

移动次数: 0

最坏情况(逆序)

$$\text{比较次数: } \sum_{i=1}^{n-1} (n - i) = \frac{1}{2}(n^2 - n)$$

$$\text{移动次数: } 3 \sum_{i=1}^{n-1} (n - i) = \frac{3}{2}(n^2 - n)$$

综上

冒泡排序

最好  $O(n)$

最坏  $O(n^2)$

平均  $O(n^2)$

需要辅助空间一个  $temp$ ,  $O(1)$

冒泡排序是稳定的

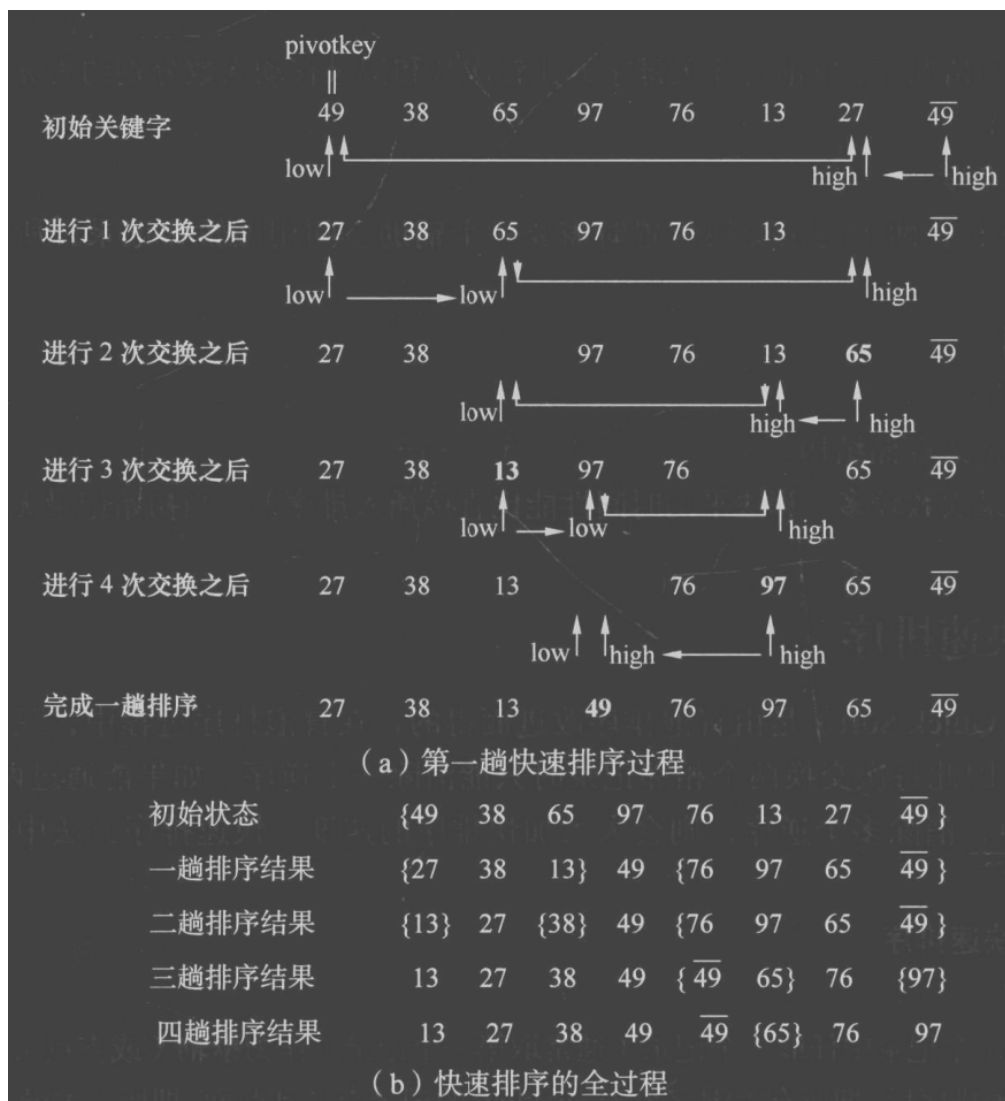
### - 快速排序

基本思路:

选取序列  $L$  中第一个元素  $a_1$  作为  $pivot$ , 依此扫描序列, 如果  $n_2$  小于  $n_1$  则排在  $n_1$  后面, 反之排在前面

此时以  $pivot$  为中心, 序列被分成两个子序列:  $L_1, L_2$

依此对  $L_1, L_2$  进行取  $pivot$  操作, 以此类推直到  $L_1, L_2$  中只有一个元素, 此时  $L$  为有序序列



代码实现:

```
//找pivot
int Partition(RecordList *L, int left, int right) {
    L->r[0] = L->r[left];

    while (left < right) {
        while (left < right && L->r[0].key <= L->r[right].key)
            // 必须<=, 如果不加等于号, 则死循环
            right--;
        L->r[left] = L->r[right];
        while (left < right && L->r[0].key >= L->r[left].key)
            left++;
        L->r[right] = L->r[left];
    }
    //此时 left和right重叠
    L->r[left] = L->r[0];
    return left;
}

//排序模型
void QSort(RecordList *L, int left, int right) {
    if (left < right) {
        //获取pivot位置
        int pivot = Partition(L, left, right);
        // pivot把序列分成两块, 并分别递归
        QSort(L, left, pivot - 1);
    }
}
```

```

    QSort(L, pivot + 1, right);
}
}
// 为了方便, 封装函数
void QuickSort(RecordList *L) {
    QSort(L, 1, L->length);
}

```

快速排序算法效率:

$QSort() : O(\log_2^n)$

$Partition() : O(n)$

平均效率为  $O(n(\log_2^n))$

快速排序不是原地排序

需要借助递归来实现, 调用栈

平均情况下需要  $O(\log n)$  的栈空间

最快情况:  $O(n)$

快速排序不是一种稳定排序

若对(90, 85, 79, 74, 68, 50, 46)或(46, 50, 68, 74, 79, 85, 90)进行快速排序

以  $pivot$  为中心, 必然有一侧的子序列个数为0, 那么此时退化成冒泡排序

所以快速排序不适用于原本有序或基本有序的序列

总结

$pivot$  的选取直接影响快排性能

数据次序越乱, 快排越快, 效率越高

快速排序不是自然排序方法

改变  $pivot$  的选取方法, 至多只能改变算法平均情况下的效率, 无法改变最快情况下的效率

即最坏情况  $O(n^2)$

## - 选则排序

|        |      |     |     |             |           |     |     |    |
|--------|------|-----|-----|-------------|-----------|-----|-----|----|
| 初始关键字  | 49   | 38  | 65  | 97          | <u>49</u> | 13  | 27  | 76 |
| 一趟排序结果 | (13) | 38  | 65  | 97          | <u>49</u> | 49  | 27  | 76 |
| 二趟排序结果 | (13  | 27) | 65  | 97          | <u>49</u> | 49  | 38  | 76 |
| 三趟排序结果 | (13  | 27  | 38) | 97          | <u>49</u> | 49  | 65  | 76 |
| 四趟排序结果 | (13  | 27  | 38  | <u>49</u> ) | 97        | 49  | 65  | 76 |
| 五趟排序结果 | (13  | 27  | 38  | <u>49</u>   | 49)       | 97  | 65  | 76 |
| 六趟排序结果 | (13  | 27  | 38  | <u>49</u>   | 49        | 65) | 97  | 76 |
| 七趟排序结果 | (13  | 27  | 38  | <u>49</u>   | 49        | 65  | 76) | 97 |

图 8.6 简单选择排序过程

```

void SelectionSort(RecordList *L) {
    int i, j;
    for (i = 1; i < L->length; i++) {
        int k = i; // k记录最大值或最小值
        for (j = i + 1; j <= L->length; j++) {

```

```

        if (L->r[k].key < L->r[j].key) //从大到小排序
            k = j;                    // k记录最大值
    }

    if (k != i) { //如果k!=i 说明有值比i大
        SortType temp = L->r[i].key;
        L->r[i].key = L->r[k].key;
        L->r[k].key = temp;
    }
}
}
}

```

## 测试代码

```

#include "ExchangeSort.h"

void input(RecordList *RL);

int main(void) {
    RecordList RL;
    RL.length = 10;
    input(&RL);
    SelectionSort(&RL);

    for (int i = 1; i <= RL.length; i++) {
        printf("%d ", RL.r[i].key);
    }

    system("pause");
    return 0;
}
// 输入数据: 49 38 65 97 76 13 27 49 55 4

```

## 算法效率分析:

时间复杂度  
 记录移动次数  
 最好情况 : 0  
 最坏情况 :  $3(n - 1)$   
 比较次数

无论待排序处于什么状态，选则排序所需进行的比较次数都相同

$$\sum_{i=1}^{n-1} (n - i) = \frac{n}{2} (n - 1) \approx \frac{n^2}{2}$$

## 算法特点

- 就选则排序本身来讲，是一种稳定的排序方法，稳定取决于是否在在比较时加入等号
- 可用于链式存储结构
- 移动记录次数较少，当每一记录占用空间较多时，此方法比直接插入排序块

## - 堆

### 堆定义

若 $n$ 个元素的序列为 $\{a_1, a_2, \dots, a_n\}$

满足如下条件

$$\begin{cases} a_i \leq a_{2i} \\ a_i \leq a_{2i+1} \end{cases} \text{ (小根堆)} \text{ 或 } \begin{cases} a_i \geq a_{2i} \\ a_i \geq a_{2i+1} \end{cases} \text{ (大根堆)}$$

从上述定义可知，堆实质上就是一个完全二叉树：二叉树中任意非叶子节点均小于(大于)它的孩子节点

如下图



### 定理1:

若有数列 $a$ 有 $n$ 个元素，若按照按下标 $1 \dots n$ 存入一颗树中，则此颗树为完全二叉树

### 定理2:

若有数列 $a$ 有 $n$ 个元素，若按照按下标 $1 \dots n$ 存入一颗完全二叉树中，令 $s = n/2$

$s + 1, s + 2, \dots, n$ 为堆

*Proof:*

根据完全二叉树的性质可知, $n/2$ 为序号最大的非叶子节点

叶子节点本身为堆，所以 $s + 1$ 到 $n$ 为堆(*By induction*)

### 初始化堆

若在输出堆顶的最小值(最大之后)，使剩余 $n - 1$ 个元素的序列重新组成一个堆

则得到 $n$ 个元素中的次小值(次大值)

对 $n - 1$ 执行如上操作，得到一个有序序列，此过程为堆排序

## 堆调整

根据定理2可知,  $s = n/2 + 1$  到  $n$  为堆, 那么只需要判断  $s$  到  $n$  是否为堆  
 如果  $s$  到  $n$  不为堆, 选取  $\max(2s, 2s + 1)$  和  $s$  交换, 此时  $s, 2s, 2s + 1$  为堆  
 但是交换了  $s$  和  $\max(2s, 2s + 1)$ , 无法保证交换后的序列是否为堆, 所以还要继续调整  
 令  $s = \max(2s, 2s + 1)$  继续调整, 直到  $s > n$

```
void HeapAdjust(RecordList *L, int s, int n) {
    RecordType temp = L->r[s]; // temp暂存, 用来作哨兵

    for (int j = 2 * s; j <= n; j *= 2) {
        //此处 j<n 确保了如果j是序列最后的元素不进行比较
        if (j < n && L->r[j].key < L->r[j + 1].key)
            j++; // j记录最大值
        //如果j的key<=temp 说明temp为根的大根堆
        if (L->r[j].key <= temp.key)
            break;
        //因为j.key > s.key 所以把j赋给s
        L->r[s] = L->r[j];
        //!! 注意并不需要更改j
        // 因为当发生堆调整时j会赋值给s, 并且s=j,即再次循环发生堆调整时, s变动, 也就是说上一个j发生变动
        //所以不用特地给j赋值。
        s = j;
    }
    //当循环结束再给s赋值, 也就是给上一个j赋值
    L->r[s] = temp;
}
```

## 堆排序

堆初始化虽然完成并且有序, 但是  $s + 1$  到  $n$  并不是有序  
 此时堆顶元素为  $\max$ , 交换堆顶元素和  $n$  号元素  
 并且对  $1$  到  $n - 1$  进行堆调整, 此时  $n$  号元素为  $\max$   
 依此类推, 继续交换堆顶和  $n - 1$ , 并对  $1$  到  $n - 2$  进行堆调整  
 直到  $n = 2$

```
void HeapSort(RecordList *L) {
    InitHeap(L);
    for (int i = L->length; i > 1; i--) {
        RecordType temp = L->r[1];
        L->r[1] = L->r[i];
        L->r[i] = temp;
        HeapAdjust(L, 1, i - 1);
    }
}
```

## 堆排序算法效率分析



初始化堆： $O(n)$

交换堆顶元素和 $n$ 元素，并重新堆调整： $O(n \log^n)$

所以堆排序 =  $O(n) + O(n \log^n) = O(n \log^n)$

具体推导过程在书中(第二版)254页

堆排序的特点

- 不是稳定排序
- 只能用于顺序结构，不能用于链式结构
- 堆排序在最坏情况下时间复杂度也为 $O(n \log^n)$ ，无论待排序序列是正序还是逆序都一样
- 初始化堆时，需要比较的次数较多，因此记录较少时不宜采用。堆排序在最坏情况下 $O(n \log^2)$ ，相对于快速排序最坏情况 $O(n^2)$ 而言是一个优点，当记录较多时效率高

## • 其他类型排序

### - 并归排序 Merge Sort

基本思想

若有序列 $a_1 a_2 \dots a_{mid} \dots a_{n-1} a_n$

其中 $a_1 a_2 \dots a_{mid}$ 为非递减序列， $a_{mid+1} \dots a_{n-1} a_n$ 也为非递减序列

依此比较 $a_1, a_{mid+1}$ 取较小值放入新建序列 $\{a'\}$

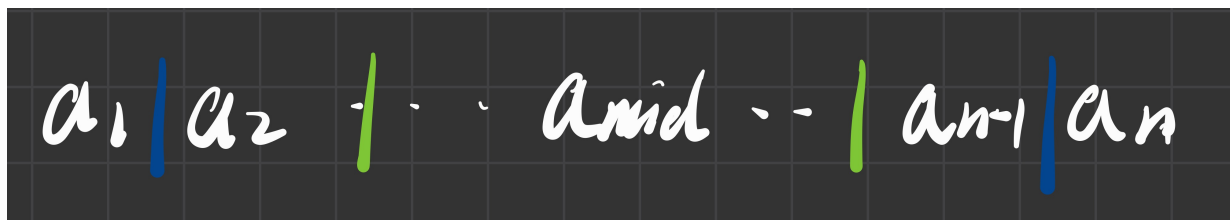
依此类推得到的序列 $\{a'\}$ 为有序序列

但是如果一个杂乱无章的序列应该如何应用Merge Sort?

若有序列 $a_1 a_2 \dots a_{mid} \dots a_{n-1} a_n$

依此对该序列进行二分，直到获得只有一个元素的子序列

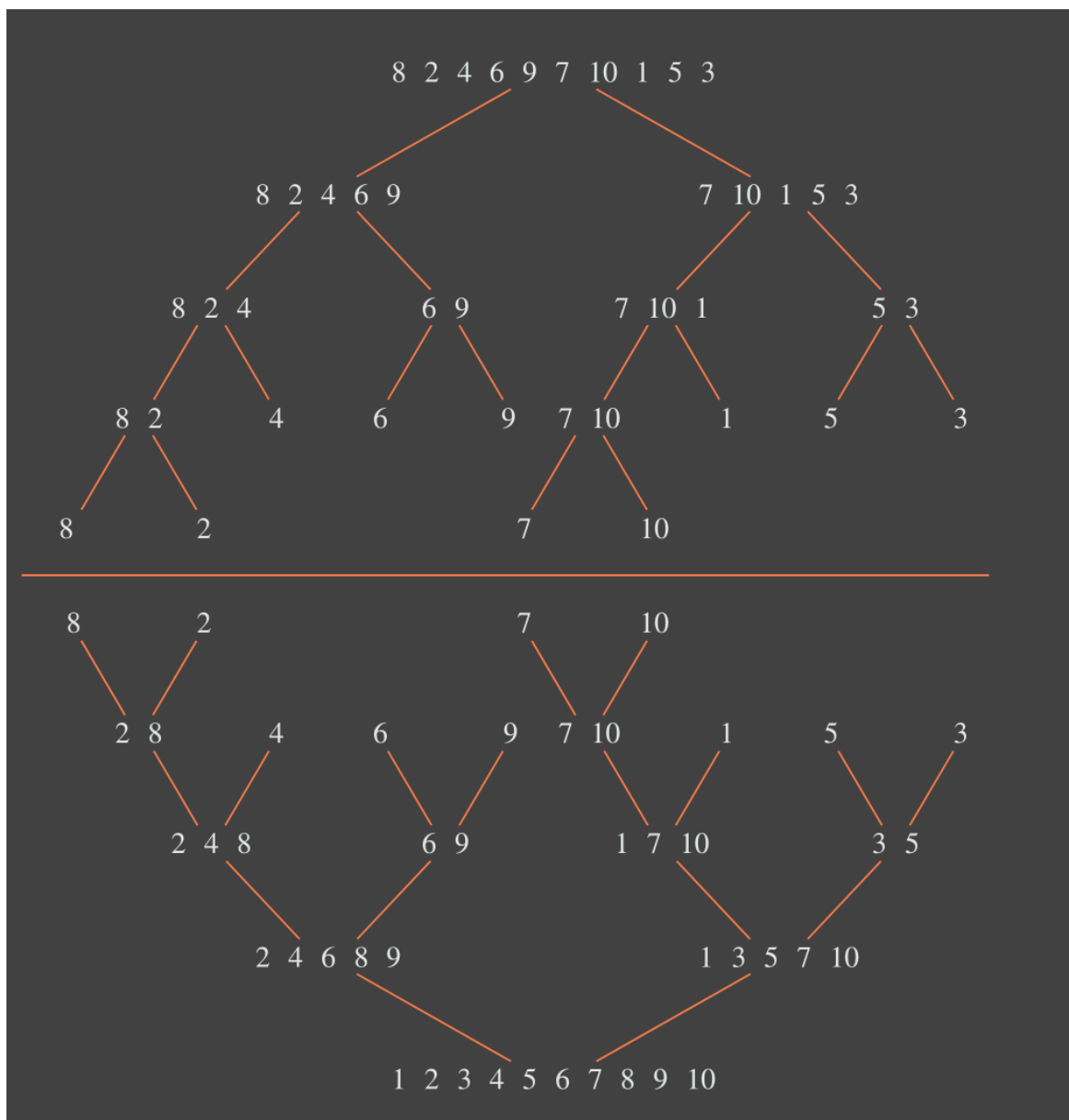
如下图



比较 $a_1$ 和 $a_2$ ，较小值放入 $\{a'\}$ 充当 $a'_1$ ，较大值则充当 $a'_2$ ，此时 $a'_1, a'_2$ 为有序序列

按照此方法递归，可以得到有序序列

例子



实现代码

### 合并两个有序序列

```
//令R的1到mid为有序序列，mid+1到n为有序序列
void Merge(RecordType *R, RecordType *Temp, int left, int mid, int right) {
    int i = left, j = mid + 1, k = left;
    while (i <= mid && j <= right) {
        //利用三目运算符简化语句
        // <= 保证了稳定性
        Temp[k++] = (R[i].key <= R[j].key ? R[i++] : R[j++]);
    }

    while (i <= mid)
        Temp[k++] = R[i++];
    while (j <= right)
        Temp[k++] = R[j++];
}
```

## 分割序列并进行合并

```
void MSort(RecordType *R, RecordType *Temp, int left, int right) {
    //递归函数的出口为left = right = 1 和 left = right = mid+1
    if (left >= right)
        return;

    int mid = (left + right) >> 1;
    MSort(R, Temp, left, mid);
    MSort(R, Temp, mid + 1, right);
    Merge(R, Temp, left, mid, right);
    /// 注意：此函数的核心语句为下行的循环
    /// 每一次递归都需要更新R数组，因为进行栈底递归时，R必须为以mid为中心，两侧都是有序序列
    for (int i = left; i < right + 1; ++i) {
        R[i] = Temp[i];
    }
}
```

## 函数封装

```
void MergeSort(RecordList *L) {
    RecordType Temp[L->length + 1];
    MSort(L->r, Temp, 1, L->length);
}
```

## 算法效率

时间复杂度： $O(n \log^n)$

空间复杂度： $O(n)$

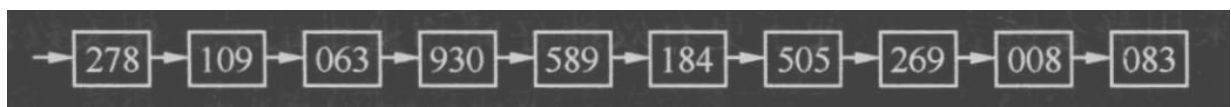
是稳定排序

可以用于链式存储结构，且不需要附加存储空间，但递归的实现仍需要开辟相应的工作栈

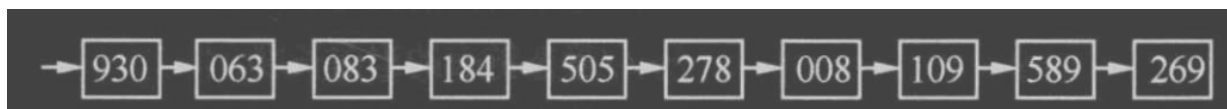
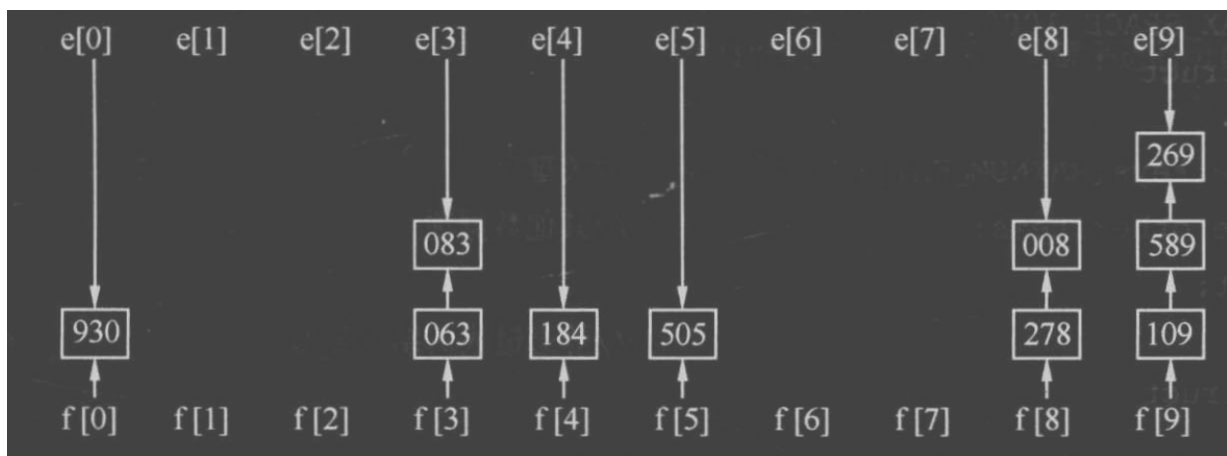
## - 基数排序

前面的算法都是基于比较，而基数排序则不需要比较，通过关键字中的信息进行分类，进行**分配**和**采集**来实现排序

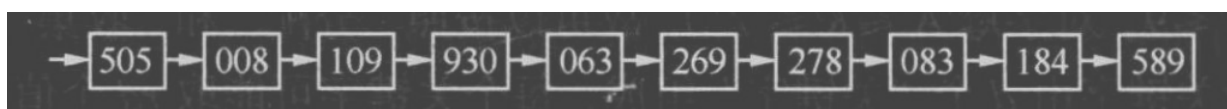
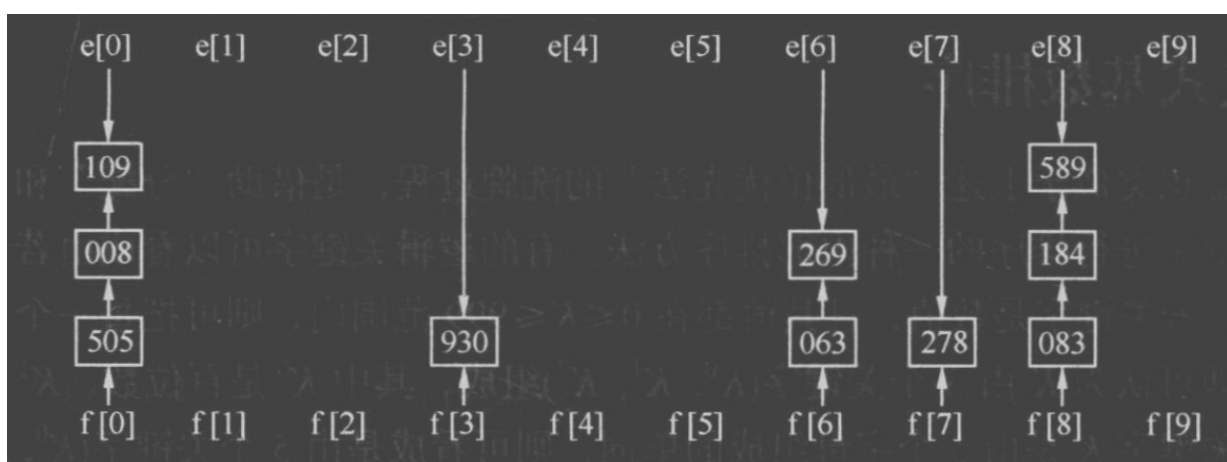
如下图



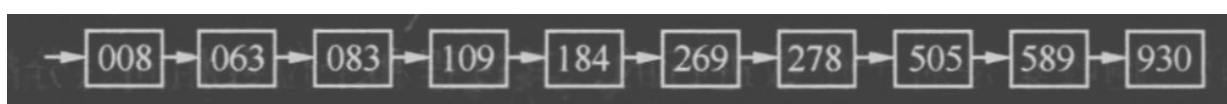
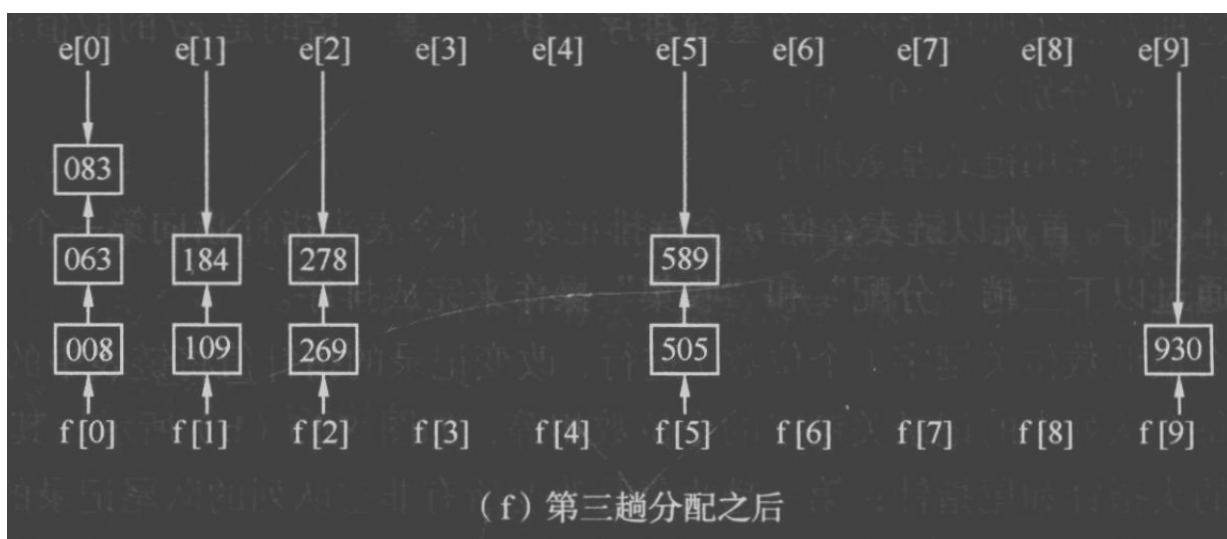
先按照个位排序



再按照十位排序



再按百位排序



## 数据类型定义

采用静态链表来对3位数字排序

```
#define MAXBIT 3 //排序的关键字为3位
#define RADIX 10 //对10进制进行排序
#define MAX_SPACE 100 //最多可以有99个待排序元素，因为有一个头节点
struct SLCell { //元素类型
    SortType keys[MAXBIT]; //存储个位，十位，百位
    SortOther other; //其他信息
    int next; //存放下一个元素在数组中的位置
};

struct SLList {
    SLCell r[MAX_SPACE]; // r[0]不存放数据，类似于链表的头指针
    int bitnumber; //表示此静态链表对n位数排序
    int length; //表中有效元素个数
};

typedef int RadixArr[RADIX]; //用于创建first, end 数组
```

## 分配函数

```
void Distribute(SLCell *r, int i, RadixArr first, RadixArr end) {
    // r表示 SLCell数组的首地址，i=0,i=1,i=2 分别表示对百位，十位，个位进行分配
    // first数组存放首个被分配的下标，end存放first指向的最后元素
    memset(first, 0, sizeof(int) * RADIX); //初始化为0
    memset(end, 0, sizeof(int) * RADIX);
    for (int p = r[0].next; p; p = r[p].next) {
        //因为r[0]为头指针，所以p指向表中第一个元素
        int j = r[p].keys[i]; // j为下标，等式右边则表示映射关系
        //如果first[j]==0说明first指向任何元素，直接把p赋给first[j]，
        if (!first[j])
            first[j] = p;
        else {
            // first[j]已经有指向，那么需要找到end[j]，并把它们连起来
            r[end[j]].next = p;
        }
        //因为p指向新加入的元素，所以最后一个元素变成p
        end[j] = p;
    }
}
```

## 收集函数

```
void Collect(SLCell *r, int i, RadixArr first, RadixArr end) {
    //此时分配已经完成，需要做的是按顺序把分配的元素连起来，即收集
    int j = 0;
    //寻找第一个非空的first子表
```

```

while (!first[j])
    j++;
//此时j指向第一个非空子表
r[0].next = first[j]; //让头指针指向此子表
int tail = end[j];    // tail代表此子表最后元素的下标
//寻找第2个非空子表，依此类推，直到j>=Radix
for (j = j + 1; j < RADIX; j++) {
    if (!first[j])
        continue;           //如果子表为空则跳过
    else {                   //当不为空时
        r[tail].next = first[j]; //让上一个子表的最后一个元素指向first[j]
        tail = end[j];         //此时更新尾部下标
    }
}
//当下面循环结束后，说明收集完毕
r[tail].next = 0;
}

```

## 排序函数

```

void RadixSort(SLList *L) {
    //创建first,end数组，不需要初始化，因为Distrubute(函数会进行初始化)
    RadixArr first, end;
    //因为是静态链表，需要更新next
    for (int i = 0; i < L->length; ++i) {
        L->r[i].next = i + 1;
    }
    L->r[L->length].next = 0; //设置结束表示0
    //因为是对三位数进行分配，依此对个位，十位，百位进行分配并收集
    for (int i = L->bitnumber - 1; i >= 0; --i) {
        Distrubute(L->r, i, first, end);
        Collect(L->r, i, first, end);
    }
}

```

## 初始化和输出函数

```

void Init_Radix_3(SLList *L, int n) {
    int hundreds, tens, ones;
    printf("Please input %d intergers(tree digits):", n);
    for (int i = 1; i < n + 1; ++i) {
        int value;
        scanf(" %d", &value);
        ones = value % 10;
        tens = (value / 10) % 10;
        hundreds = value / 100;
        L->r[i].keys[0] = hundreds;
        L->r[i].keys[1] = tens;
        L->r[i].keys[2] = ones;
    }
}

```

```

    }
    L->length = n;
    L->bitnumber = 3;
}

void Display_Radix(SLList *L) {
    for (int p = L->r[0].next; p != NULL; p = L->r[p].next) {
        for (int i = 0; i < L->bitnumber; i++) {
            printf("%d", L->r[p].keys[i]);
        }
        printf(" ");
    }
}
}

```

## 算法效率

令基数为 $r$ , 有 $n$ 个记录, 每个记录含有 $d$ 个关键字, 则分配需要 $O(nd)$ , 收集则需要 $O(rd)$

所以 $O(d(n + r))$

需要两个长度为 $Radix$ 的 $first, end$ 数组, 且还增加了个 $n$ 个 $next$ 元素

所以空间复杂度:  $O(n + 2r) \approx O(n + r)$

算法特点:

- 稳定排序
- 可用于链式存储结构
- 只要基数选取合适, 时间复杂度是线性的可以达到 $O(n)$
- 有严格的使用要求: 需要直到各级关键字的主次关系和取值范围

## 排序总结

| 排序方法   | 最好情况          | 最坏情况          | 平均情况          | 空间复杂度       | 稳定性 |
|--------|---------------|---------------|---------------|-------------|-----|
| 直接插入排序 | $O(n)$        | $O(n^2)$      | $O(n^2)$      | $O(1)$      | 稳定  |
| 折半插入排序 | $O(n \log^n)$ | $O(n^2)$      | $O(n^2)$      | $O(1)$      | 稳定  |
| 希尔排序   |               |               | $O(n^{1.3})$  | $O(1)$      | 不稳定 |
| 冒泡排序   | $O(n)$        | $O(n^2)$      | $O(n^2)$      | $O(1)$      | 稳定  |
| 简单选择排序 | $O(n^2)$      | $O(n^2)$      | $O(n^2)$      | $O(1)$      | 稳定  |
| 快速排序   | $O(n \log^n)$ | $O(n^2)$      | $O(n \log^n)$ | $O(\log^n)$ | 不稳定 |
| 堆排序    | $O(n \log^n)$ | $O(n \log^n)$ | $O(n \log^n)$ | $O(1)$      | 不稳定 |
| 归并排序   | $O(n \log^n)$ | $O(n \log^n)$ | $O(n \log^n)$ | $O(n)$      | 稳定  |
| 基数排序   | $O(d(n + r))$ | $O(d(n + r))$ | $O(d(n + r))$ | $O(n + r)$  | 稳定  |

按照时间性能来区分：

$O(n \log^n)$ 有：快速排序，归并排序，堆排序，其中快速排序最好

$O(n^2)$ 有：直接插入排序，冒泡排序，简单选择排序，其中直接插入最好

特别是对于关键字近似有序的记录

$O(n)$ 只有：基数排序

当待排记录有序时，直接插入排序和冒泡排序能达到 $O(n)$ ，而对于快速排序而言，这是最不好的情况

此时快速排序 $O(n^2)$

简单选择排序，堆排序，归并排序的效率并不能根据关键字的分布而改变

按空间性能来区分：

所有简单排序方法(直接插入，冒泡，简单选择排序)和堆排序为 $O(1)$

快速排序为 $O(\log^n)$ ，需要借助栈

归并排序需要辅助空间最多， $O(n)$

链式基数排序需要 $first$ ， $end$ 数组和 $next$ 变量， $O(n + r)$

按稳定性来区分：

快速排序和堆排序不是稳定的方法